UNiCON
Unmanned & Intelligent systems CONtrol LAB

Open CV

Open Source Computer Vision

강사 소개

- **박형준 (Hyeong Jun Park)**
- **한양대학교 인공지능학과 석박통합과정**
- **지능형시스템제어 연구실 (지도교수: 강창묵 교수, https://uniconlab.wixsite.com/main)**
- **관심 분야: 인공지능(RL, VLA, LLM), 자율주행 차량**
- **프로젝트**

- 온디바이스 AI 컴퓨팅 기반 AI 융합형 모빌리티 디바이스 기술 개발 및 실증
- 다목적 개인이동형 서비스 로봇 플랫폼 기술개발 및 실증
- 환자 이승 로봇

Cotents

1. Open CV 개요

1) OpenCV란?
2) OpenCV설치(Python 기반)
3) OpenCV의주요기능
4) CVbridge 기반ROS환경및OpenCV연동

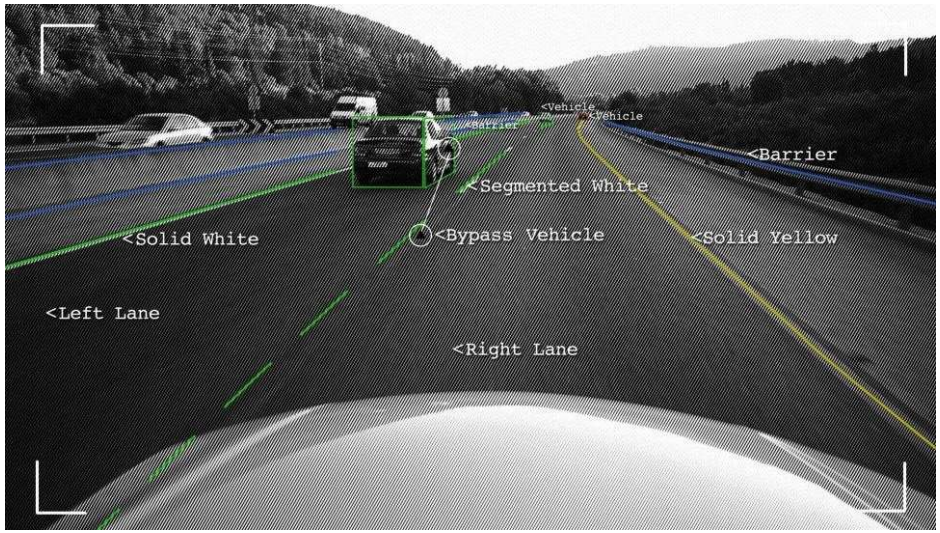
2. Open CV 실습

1) 이미지출력
2) 이미지grayscale
3) 이미지edge 검출
4) 이미지윤곽선검출
5) 이미지회전/크기조절/기울이기
6) 이미지filtering
7) 이미지속객체검출
8) 차선인식(Hough/HSV/Hough+HSV)
9) ROSGazebo 시뮬레이션기반차선인식

Camera

Camera 개요 – 자율 주행 카메라 역할과 한계

1) Camera?



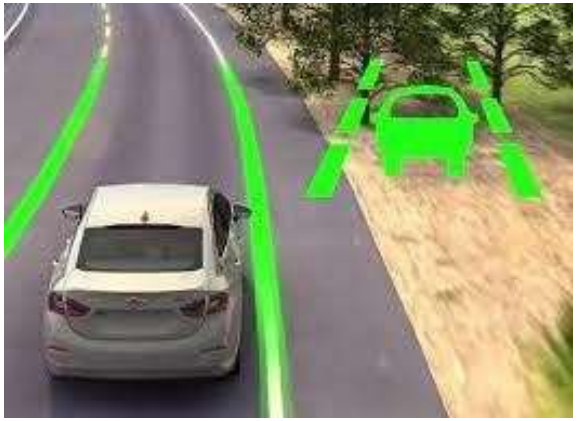
- 카메라 기술은 자율 주행 자동차에서 주변 환경을 촬영하고, 분석하는 핵심적인 센서
- 정교한 이미지 센서가 빛을 전자 신호로 변환하며, 알고리즘이 이 신호를 처리하여 각종 객체 인식을 함.

2) 인지와 물체감지

- 카메라는 자율주행 차량에서 인지를 가능하게 하며, 도로 상의 다른 차량, 보행자, 자전거, 장애물 등을 감지하고 인식함.
- 이 데이터는 차량이 주변 환경을 이해하고 잠재적인 위험을 인지하는데에 중요한 요소.

UNiCON
Unmanned & Intelligent systems CONtrol LAB

3) 차선 이탈 경고와 차선유지 ex) LDW([**Lane Departure Warning**] ,LKA[**Lane Keeping Assist**], LFA [**Lane Following Assist**]



- 카메라는 차선 이탈 경고시스템에 기여하며 차량이 의도하지 않게 차선을 이탈할 때 운전자나 자율 주행 시스템에 경고를 줌.
- 차선 유지 시스템에서 차량이 안전하게 차선을 유지하는데에 도움을 줌.

UNiCON
Unmanned & Intelligent systems CONtrol LAB

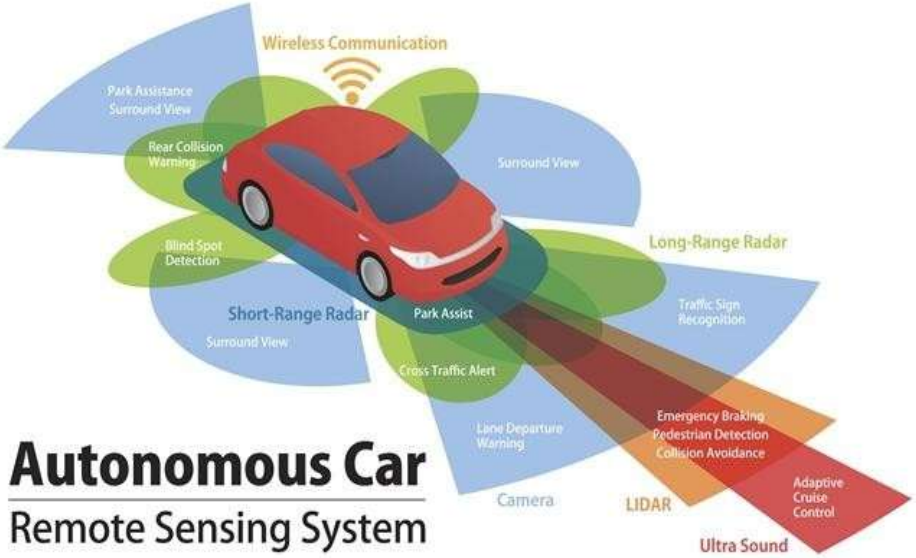
4) 교통 표지판 인식

- 카메라는 교통 표지판 인식에도 중요한 기능을 함.
- 속도 제한, 정지 표지판 ,교통 신호등 등 다양한 교통 표지판을 인식하고 해석하여 차량이 교통 규칙과 규정을 준수 할 수 있도록 도와줌.

5) 보행자 감지와 충돌 회피 (FCA: Forward Collision-Avoidance Assist)

- 보행자가 차량의 경로에 감지되면 비상 제동이나 회피 조치를 할 수 있음.
- 고급 알고리즘을 탑재한 카메라는 보행자를 정확하게 감지하고 움직임을 추정하여 회피에 중요한 역할을 함.

6) 센서 퓨전에서의 역할



- 센서 퓨전 과정에서 카메라 데이터는 LiDAR, RADAR 및 기타 센서의 입력과 결합하여, 주변 환경을 포괄적으로 인지함.
- 센서 퓨전은 인지 정확성을 향상 시켜 차량이 복잡한 운전 상황에서 더 나은 의사 결정을 내릴 수 있도록 도움.

Camera 개요 – 자율 주행 카메라 역할과 한계

카메라의 한계

https://www.youtube.com/watch?v=IQJL3htsDyQ

09:00 ~

- 카메라는 기상 조건이나 어두운 상황에서 성능이 감소하는 등의 한계를 가짐.
- 기술 회사들은 카메라 기술들을 지속적으로 개선하고 기계 학습 알고리즘등을 활용하여 이러한 도전을 해결하고 있음.

1. Open CV 개요

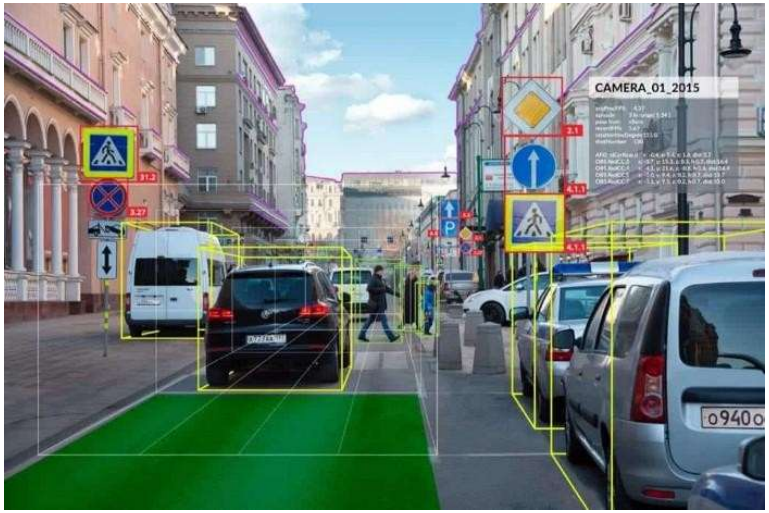
1) Open CV(Open Source Computer Vision)란?

- Open CV는 "**Open Source Computer Vision Library"**의 약자로, 이미지 및 동영상 처리에 사용되는 라이브러리

- C++, Python, Java 등 다양한 프로그래밍 언어를 지원하며, 이미지 프로세싱, 컴퓨터 비전 및 머신러닝 알고리즘에 대한 다양한 함수와 라이브러리 제공

- Open CV는 무료로 제공되며, 비즈니스 및 개인 프로젝트 모두에서 사용 가능

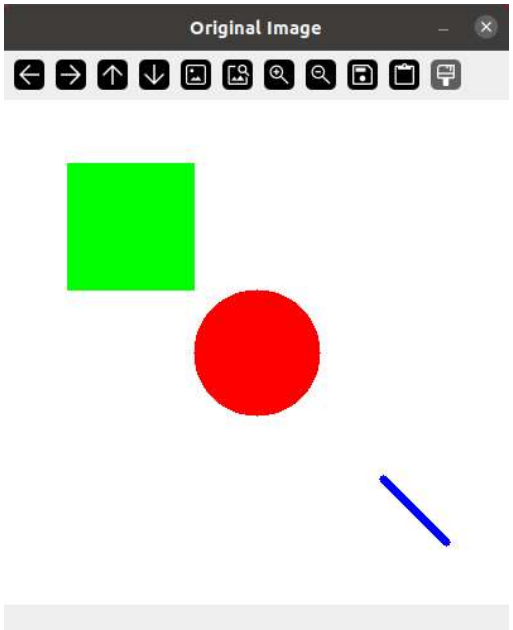
1) Open CV(Open Source Computer Vision)란?



- **Open CV**를 통해 이미지나 영상에서 **객체 검출**, **추적**, **특징 추출**, **패턴 인식** 등의 작업 수행 가능
- Open CV에서는 이미지 필터링, 색 공간 변환, 카메라 보정 등 다양한 기능을 제공
- 최근 **Open CV**와 **Deep Learning**을 결합하여 이미지 및 영상 데이터를 쉽게 처리할 수 있게 되었으며, 이를 기반으로 객체 인식 및 추적 등의 작업을 빠르게 수행할 수 있게 되었음

3) Open CV 주요 기능

- **이미지 읽기 및 쓰기**
  → 이미지를 파일에서 읽어오고, 처리된 이미지를 파일로 저장 가능

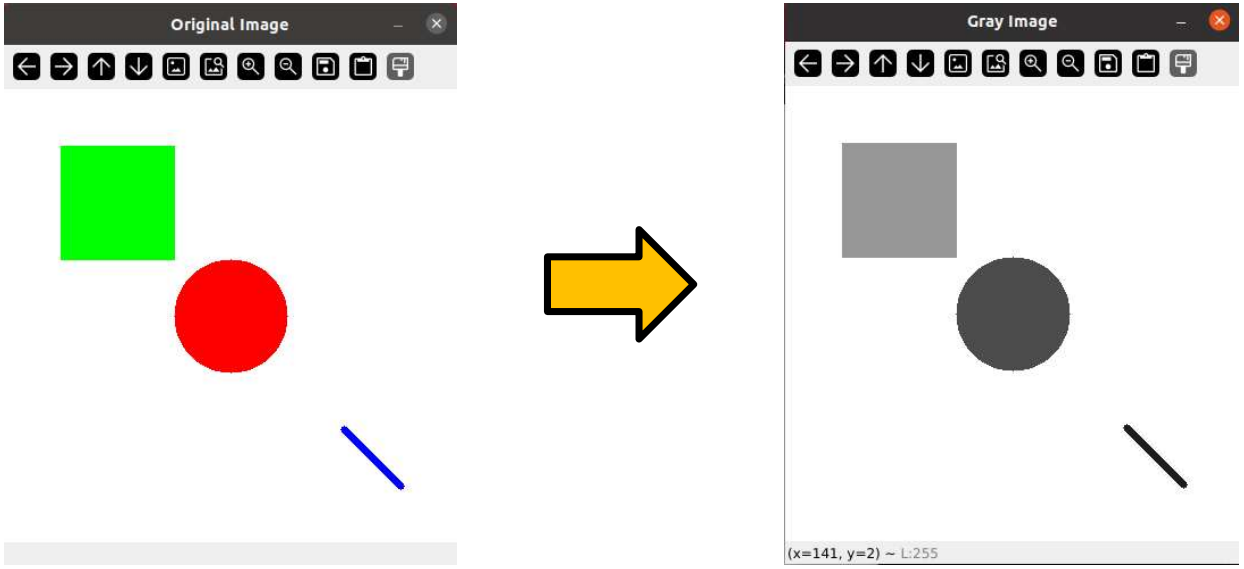


3) Open CV 주요 기능

- **이미지 변환**

  → 이미지를 다양한 색상 공간으로 변환 가능

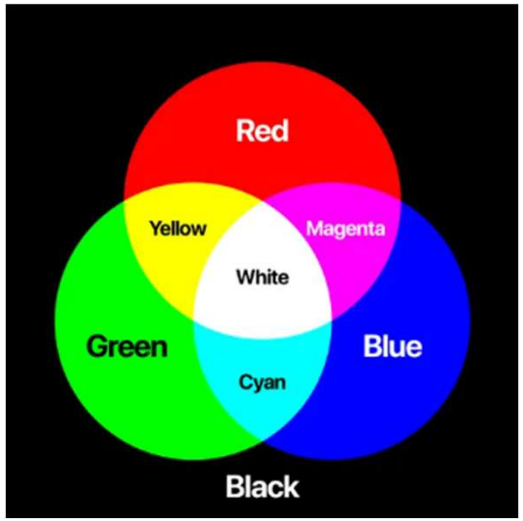
  → 다음 예시와 같이, RGB 이미지를 Gray scale 또는 HSV 색상 공간으로 변환 가능



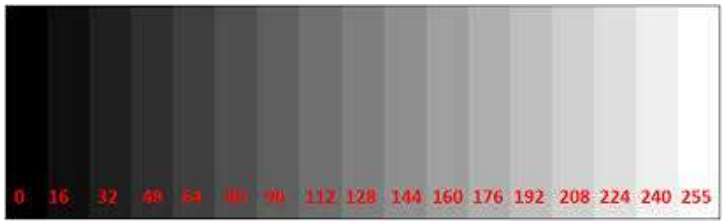
UNiCON
Unmanned & Intelligent systems CONtrol LAB

3) Open CV 주요 기능

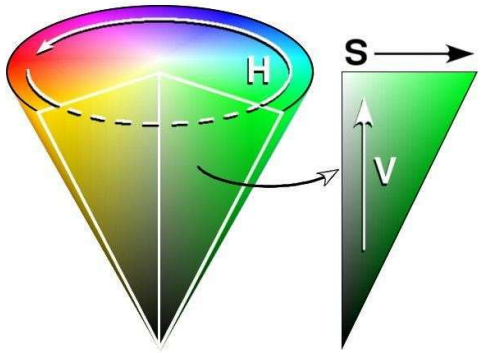
- **RGB**



- **Gray Scale**



- **HSV**

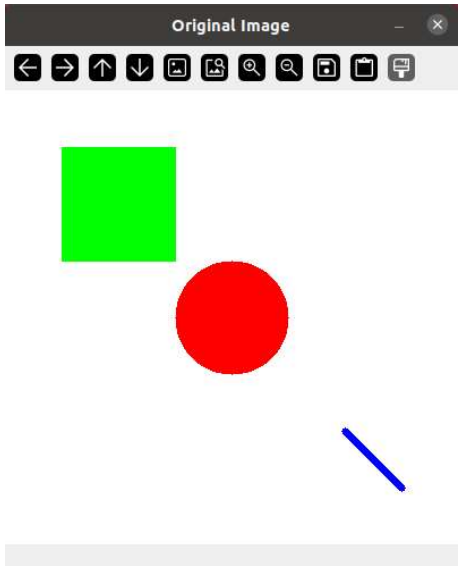


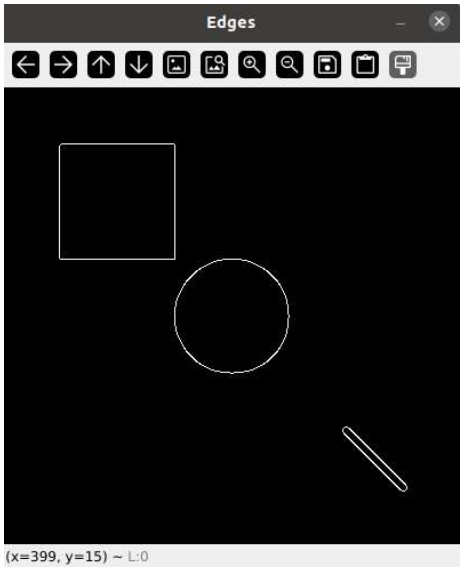
3) Open CV 주요 기능

- **Edge 검출**

  → 이미지에서 객체의 경계를 찾기 위해 edge를 검출

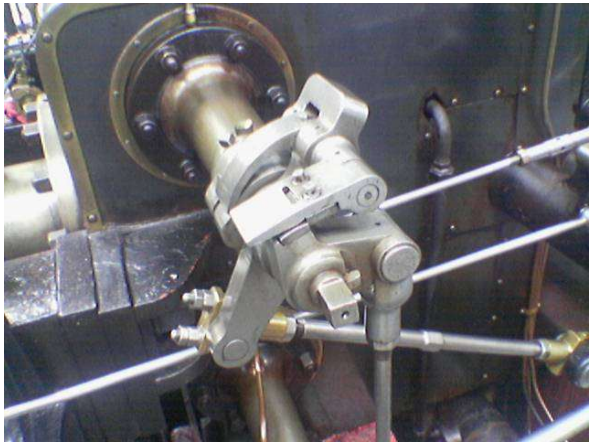
  → 대표적인 edge 검출 기법으로 'Canny edge' 검출 기법이 존재

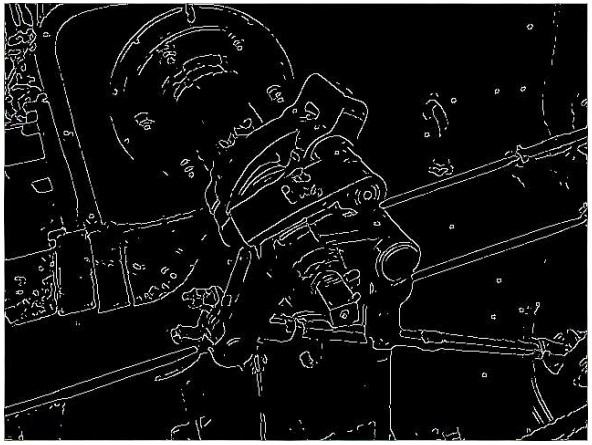


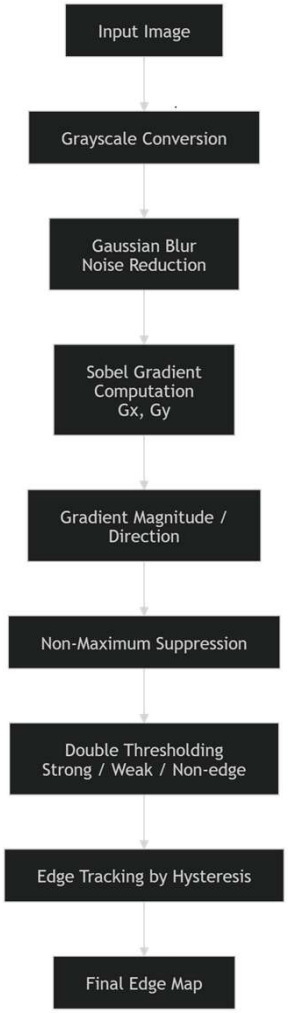


UNiCON
Unmanned & Intelligent systems CONtrol LAB

- **Canny edge detection**







$$\mathbf{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * \mathbf{A}$$

$$\mathbf{G}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * \mathbf{A}$$

ıyscale로 변환

ᴇ 검출 방지

→ Sobel Gradient Computation: x, y 방향 기울기 G_x, G_y계산, sobel filter(3 x 3) kernel

→ Gradient Magnitude / Direction: 에지 강도와 방향 계산

→ Non-Maximum Suppression: edge 후보를 걸러 실제 edge line만 남기도록 함

→ Double Thresholding: high/low 로 임계값을 설정

→ Edge Tracking by Hysteresis: weak edge 중 strong edge와 연결된 것만 유지 나머지는 제거

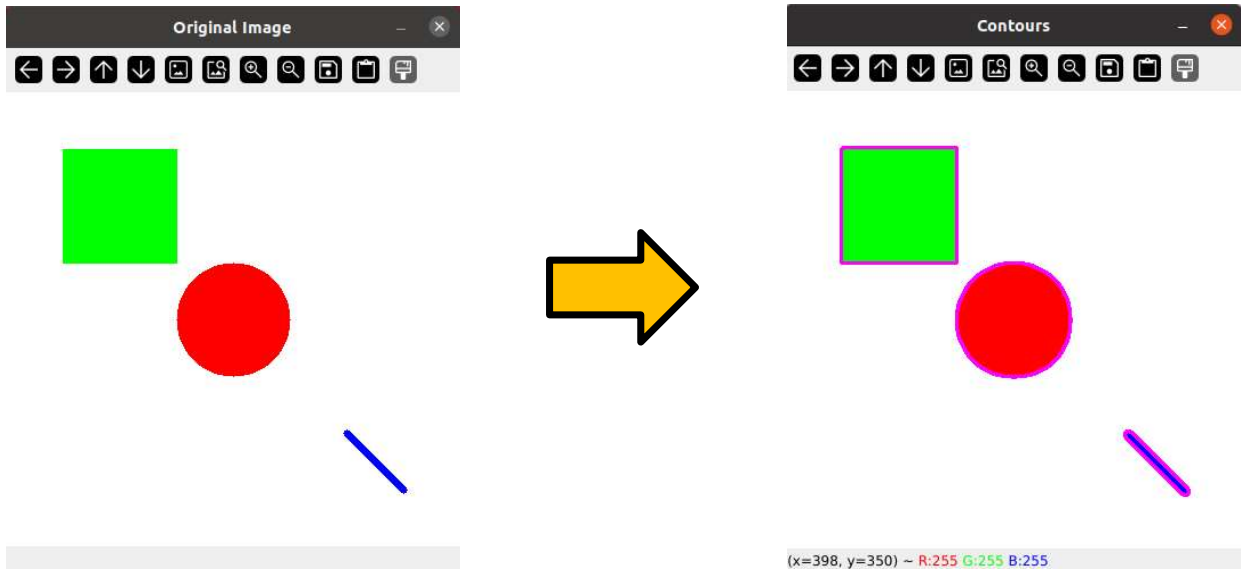
→ Final Edge Map: 최종 canny edge 결과 출력

UNiCON
Unmanned & Intelligent systems CONtrol LAB

3) Open CV 주요 기능

- **윤곽선 검출**

  → 이미지에서 객체의 윤곽선을 검출



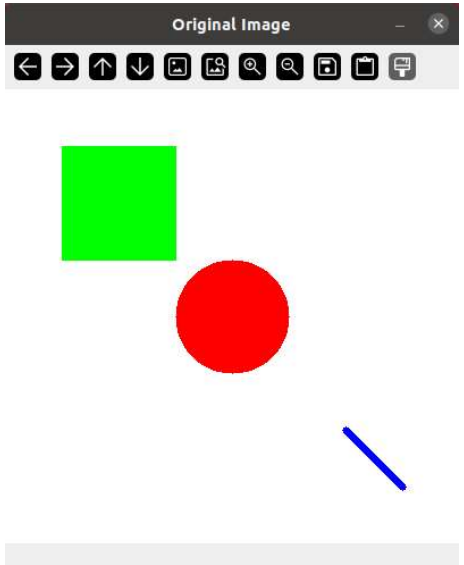
UNiCON
Unmanned & Intelligent systems CONtrol LAB

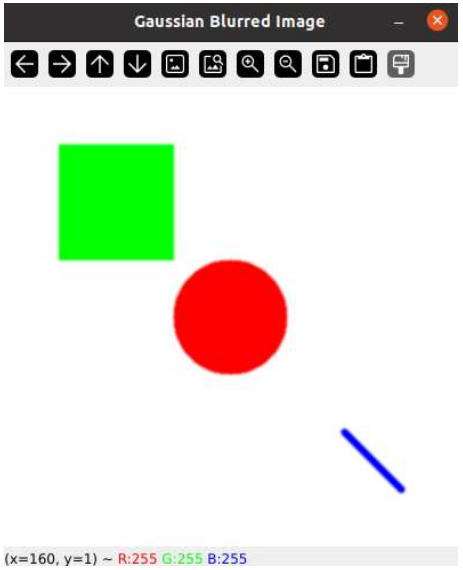
3) Open CV 주요 기능

- **이미지 필터링**

  → 이미지를 필터링하여 노이즈를 제거하거나 부드럽게 변환 가능

  → 대표적인 필터링 기법으로 'Gaussian Blur', 'Median Blur' 등이 있음

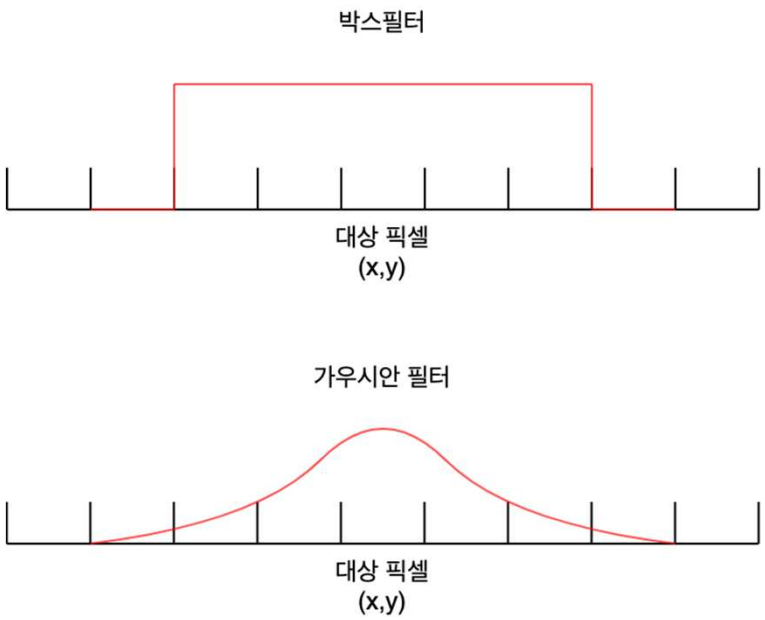




UNiCON
Unmanned & Intelligent systems CONtrol LAB

3) Open CV 주요 기능

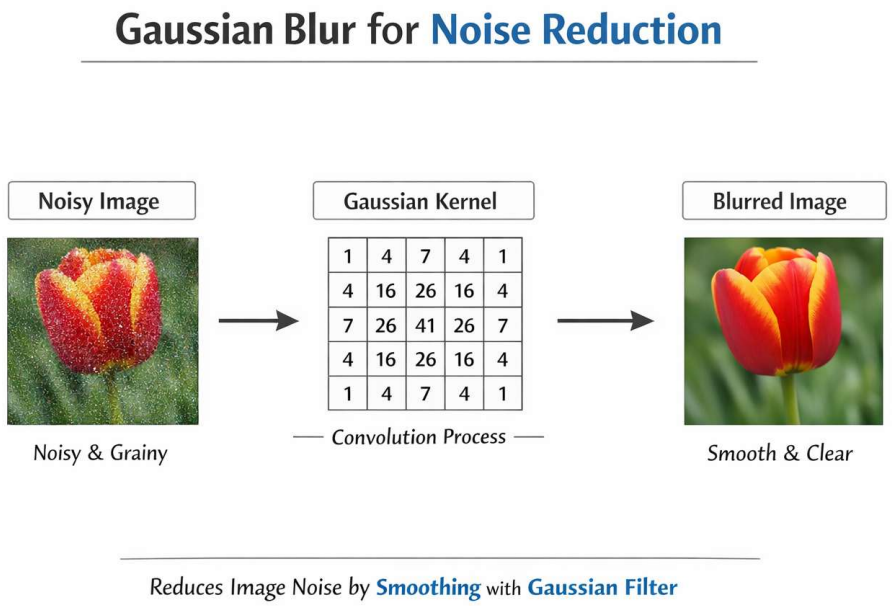
- **Gaussian Blur**

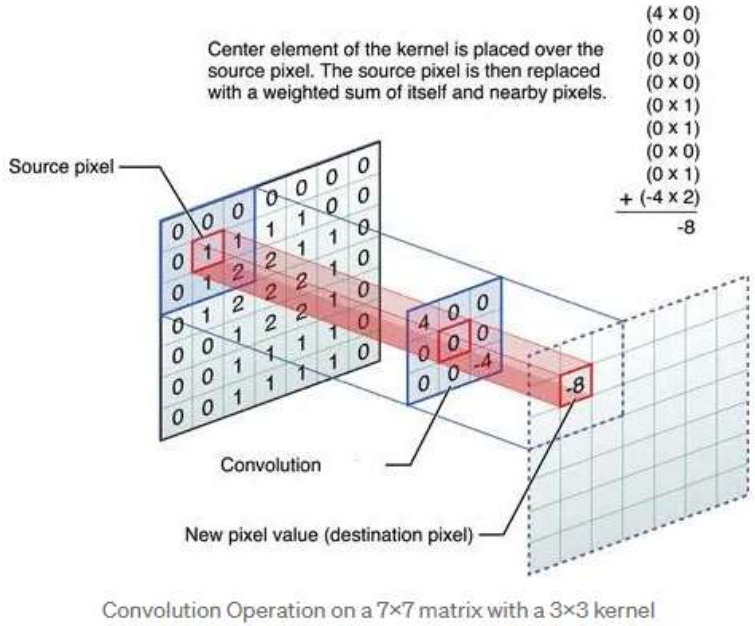


UNiCON
Unmanned & Intelligent systems CONtrol LAB

3) Open CV 주요 기능

- **Gaussian Blur**

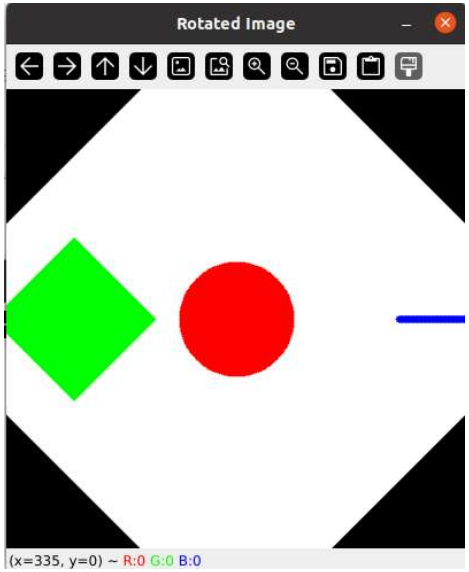




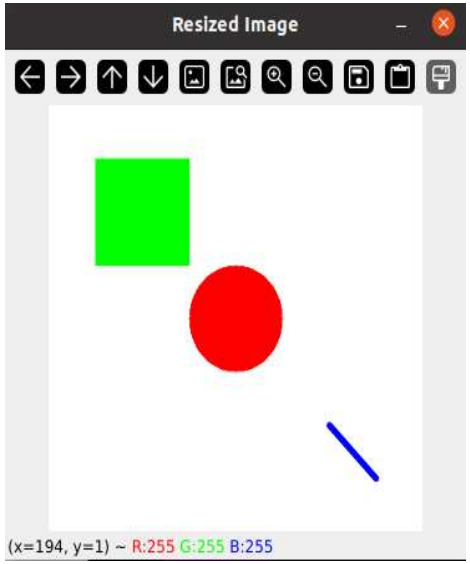
UNiCON
Unmanned & Intelligent systems CONtrol LAB

3) Open CV 주요 기능

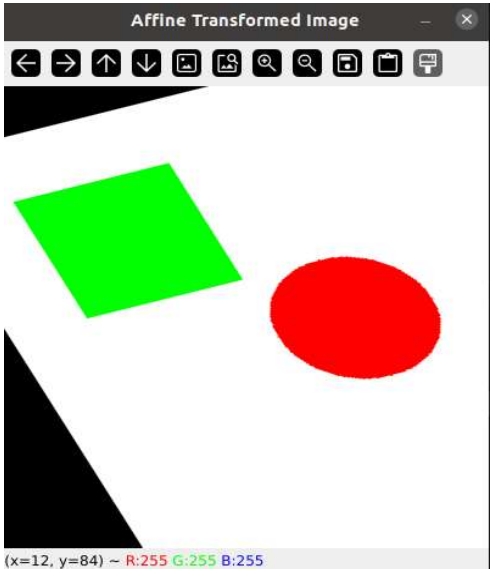
- **이미지 조정**
  → 이미지 회전, 크기 조절, 기울이기 등의 기능을 통해 이미지 데이터를 변환 가능



이미지 회전



이미지 크기 조절



이미지 기울이기

UNiCON
Unmanned & Intelligent systems CONtrol LAB

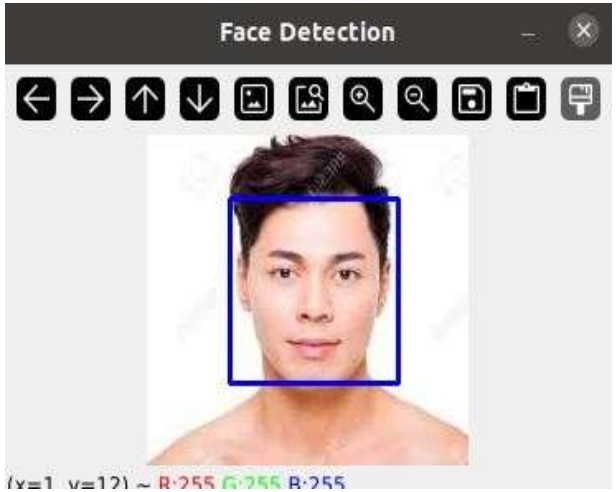
3) Open CV 주요 기능

- **객체 인식**

  → 특정 객체를 인식하고 위치를 파악 가능

  → 객체 인식을 위한 대표적인 기법으로는 'Haar Cascade' 기반 객체 인식과, Deep Learning 기반의 객체 인식 등이 존재

  → 'Haar Cascade'란, Open CV에 내장된 기능으로 얼굴 또는 객체 등을 검출 시 이용되는 머신러닝 기반의 기술을 의미



UNiCON
Unmanned & Intelligent systems CONtrol LAB

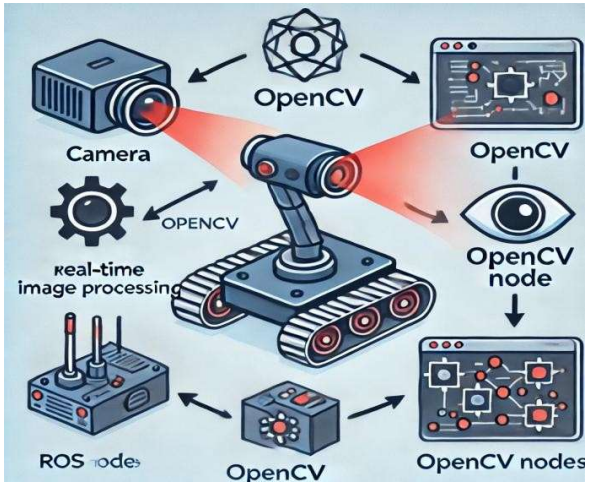
4) CV bridge 기반 ROS 환경 및 Open CV 연동

ROS + OpenCV

- **실시간 이미지 처리**
  → 로봇에 부착된 Camera 센서를 통해 수집된 이미지를 실시간으로 처리하여
  장애물 감지, 객체 추종 등 다양한 작업을 수행 가능
- **모듈성 및 재사용성**
  → ROS(Robot Operating System)의 Module(모듈)식 구조를 활용하여 Open CV 기반 이미지 처리 기능을
  독립적인 노드로 구현하고, 다양한 프로젝트에서 재사용 가능
- **커뮤니티 및 오픈 소스 지원**
  → ROS와 Open CV는 모두 오픈 소스 프로젝트로, 광범위한 커뮤니티 지원과 수 많은 자료를 제공
  → 최신 기술에서 발생할 수 있는 문제에 대한 해결이 용이하며, 이를 바탕으로 다양한 최신 기술을 편하게 사용 가능

UNiCON
Unmanned & Intelligent systems CONtrol LAB

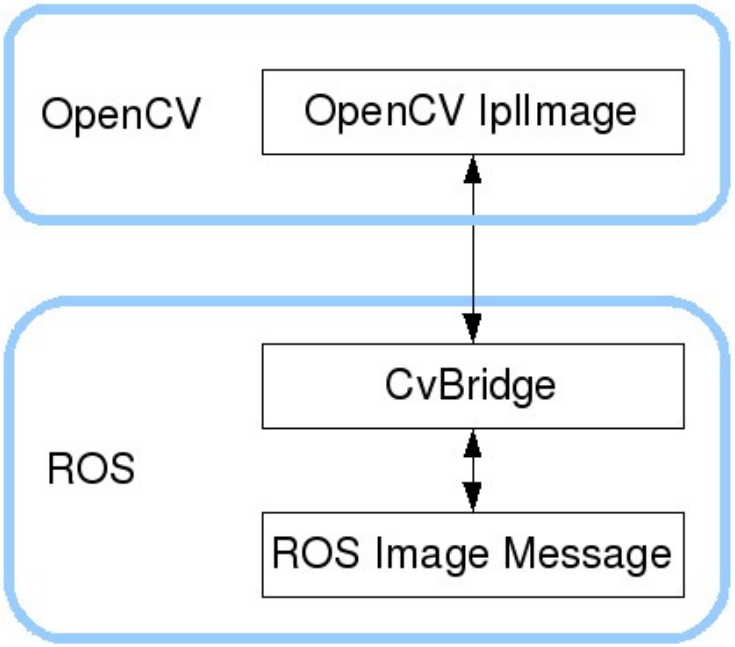
4) CV bridge 기반 ROS 환경 및 Open CV 연동



- **다양한 센서 데이터 통합**
  → Open CV를 통해 처리한 이미지 데이터와 더불어, 다양한 센서들을 결합하여 도출한 데이터를 바탕으로 더욱 정밀하게 환경 인식 수행 가능

- **시뮬레이션 및 로봇 적용**
  → Gazebo 시뮬레이터를 통해 가상 환경에서 로봇을 테스트한 뒤, 실제 로봇에 적용함으로써 테스트 및 개발 시간을 단축 가능

4) CV bridge 기반 ROS 환경 및 Open CV 연동



- **cv_bridge란?**

  → ROS에서 제공하는 패키지로, ROS 이미지 메시지와 Open CV 이미지 간의 원활한 변환을 돕는 기능

  → cv_bridge를 통해 ROS 환경의 노드가 수신한 이미지를 Open CV 형식으로 변환하여, 다양한 이미지 처리 작업을 수행 가능

4) CV bridge 기반 ROS 환경 및 Open CV 연동

- **ROS noetic 환경 기반 Open CV 설치**
  → $ pip install opencv-python (또는 $ pip3 install opencv-python)

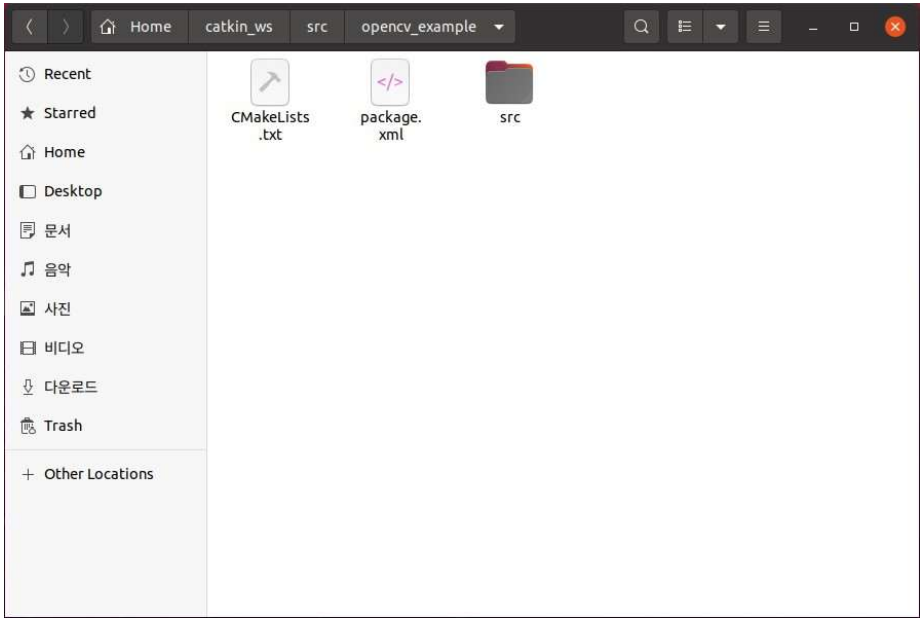
- **ROS noetic 환경에 Open CV 설치가 완료되었는지 확인**
  → $ dpkg –l | grep ros-noetic-cv-bridge

4) CV bridge 기반 ROS 환경 및 Open CV 연동

- **ROS noetic 환경에서 Camera 이미지 수신 및 Open CV 기반 처리 실습 (1/7)**

  → $ cd ~/catkin_ws/src

  → $ catkin_create_pkg opencv_example rospy std_msgs sensor_msgs cv_bridge



UNiCON
Unmanned & Intelligent systems CONtrol LAB

4) CV bridge 기반 ROS 환경 및 Open CV 연동

- **ROS noetic 환경에서 Camera 이미지 수신 및 Open CV 기반 처리 실습 (2/7)**

  → $ cd opencv_example (새롭게 생성한 패키지로 이동)

  → $ gedit CMakeLists.txt (CMakeLists 파일을 수정 → cv_bridge와 Open CV 의존성 추가)

Open | CMakeLists.txt ~/catkin_ws/src/opencv_example | Save

```
cmake_minimum_required(VERSION 3.0.2)
project(opencv_example)

## Compile as C++11, supported in ROS Kinetic and newer
# add_compile_options(-std=c++11)

## Find catkin macros and libraries
## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
## is used, also find other catkin packages
find_package(catkin REQUIRED COMPONENTS
  rospy
  std_msgs
  sensor_msgs
  cv_bridge
)

find_package(OpenCV REQUIRED)

catkin_package()

include_directories(
  ${catkin_INCLUDE_DIRS}
  ${OpenCV_INCLUDE_DIRS}
)
```

UNiCON
Unmanned & Intelligent systems CONtrol LAB

4) CV bridge 기반 ROS 환경 및 Open CV 연동

- **ROS noetic 환경에서 Camera 이미지 수신 및 Open CV 기반 처리 실습 (3/7)**
  → $ cd opencv_example (새롭게 생성한 패키지로 이동)
  → $ gedit package.xml (package.xml 파일을 수정 → cv_bridge와 Open CV 의존성 추가)

```
<buildtool_depend>catkin</buildtool_depend>
<build_depend>cv_bridge</build_depend>
<build_depend>rospy</build_depend>
<build_depend>sensor_msgs</build_depend>
<build_depend>std_msgs</build_depend>
<build_export_depend>cv_bridge</build_export_depend>
<build_export_depend>rospy</build_export_depend>
<build_export_depend>sensor_msgs</build_export_depend>
<build_export_depend>std_msgs</build_export_depend>
<exec_depend>cv_bridge</exec_depend>
<exec_depend>rospy</exec_depend>
<exec_depend>sensor_msgs</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

```
<build_depend>cv_bridge</build_depend>
<exec_depend>cv_bridge</exec_depend>

<build_depend>opencv</build_depend>
<exec_depend>opencv</exec_depend>
```

UNiCON
Unmanned & Intelligent systems CONtrol LAB

4) CV bridge 기반 ROS 환경 및 Open CV 연동

- **ROS noetic 환경에서 Camera 이미지 수신 및 Open CV 기반 처리 실습 (4/7)**

  → $ cd ~/catkin_ws/src/opencv_example/ (새롭게 생성한 패키지 내 src 폴더로 이동)

  → $ gedit test.py (src 폴더 내 python 파일 생성 후 저장)

  → $ chmod +x test.py (새롭게 생성한 python 파일에 실행 권한 부여)

```
-B650M-HDV-M-2:~$ cd catkin_ws/src/opencv_example/src/
-B650M-HDV-M-2:~/catkin_ws/src/opencv_example/src$ chmod +x test.py
```

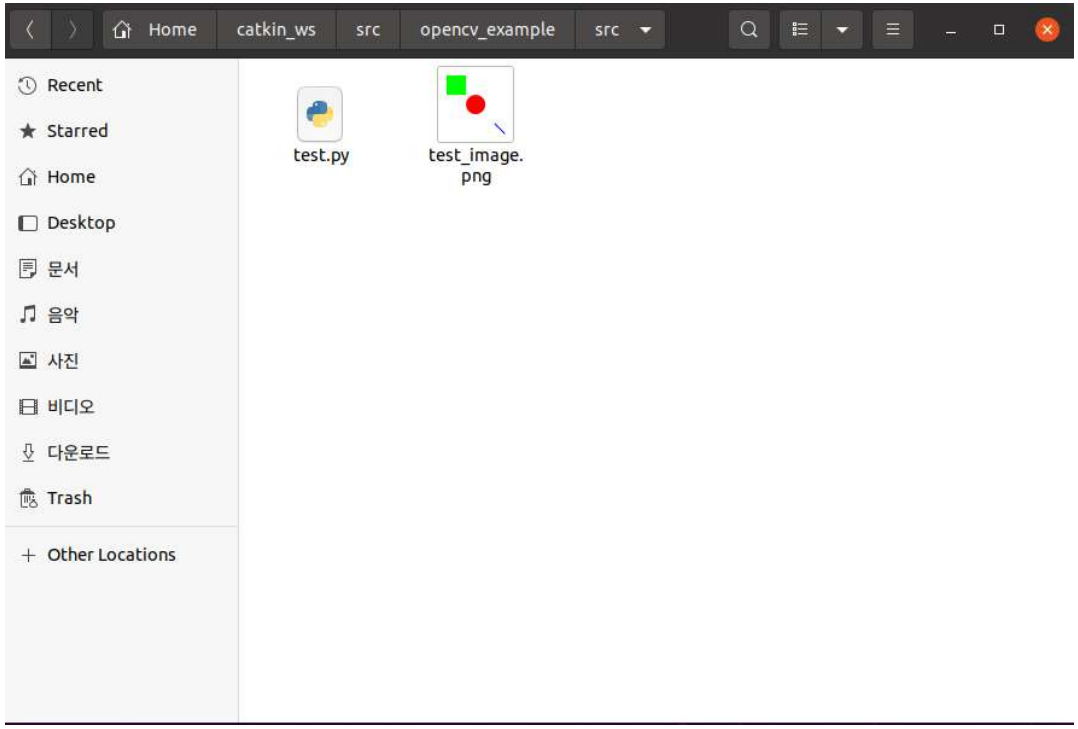
  → $ cd ~/catkin_ws/ && catkin_make (catkin_ws 빌드)

  → $ source devel/setup.bash (빌드 후, 환경 변수 재 설정)

UNiCON
Unmanned & Intelligent systems CONtrol LAB

4) CV bridge 기반 ROS 환경 및 Open CV 연동

- **ROS noetic 환경에서 Camera 이미지 수신 및 Open CV 기반 처리 실습 (5/7)**
  → Open CV 실습에 사용할 이미지를 다운로드 후, 저장



UNiCON
Unmanned & Intelligent systems CONtrol LAB

4) CV bridge 기반 ROS 환경 및 Open CV 연동

- **ROS noetic 환경에서 Camera 이미지 수신 및 Open CV 기반 처리 실습 (6/7)**

  → $ cd ~/catkin_ws/src/opencv_example/src/ (새롭게 생성한 패키지 내 src 폴더 진입)

  → $ gedit test.py (새롭게 생성한 test.py 파일에 아래와 같이 작성 후, 저장)

```python
#!/usr/bin/env python3
import cv2
import rospy


image_path = '/home/user/catkin_ws/src/opencv_example/src/test_image.png' # 이미지 경로를 설정하세요
image = cv2.imread(image_path)

# 1. 원본 이미지 표시
cv2.imshow('Original Image', image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

➔ 본인이 저장한 이미지 파일의 경로를 작성 !!

UNiCON
Unmanned & Intelligent systems CONtrol LAB

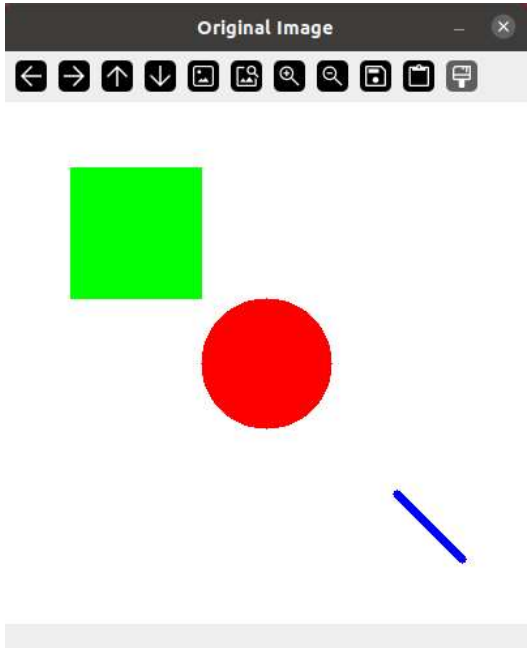
4) CV bridge 기반 ROS 환경 및 Open CV 연동

- **ROS noetic 환경에서 Camera 이미지 수신 및 Open CV 기반 처리 실습 (7/7)**

  → $ roscore (첫 번째 터미널)

  → $ cd ~/catkin_ws/src/opencv_example/src/ (두 번째 터미널)

  $ python3 test.py (두 번째 터미널)



2. Open CV 실습

UNiCON
Unmanned & Intelligent systems CONtrol LAB

1) 이미지 출력

- **Python 환경에서 Open CV를 통해 다운받은 원본 이미지를 화면에 출력하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.

test.py

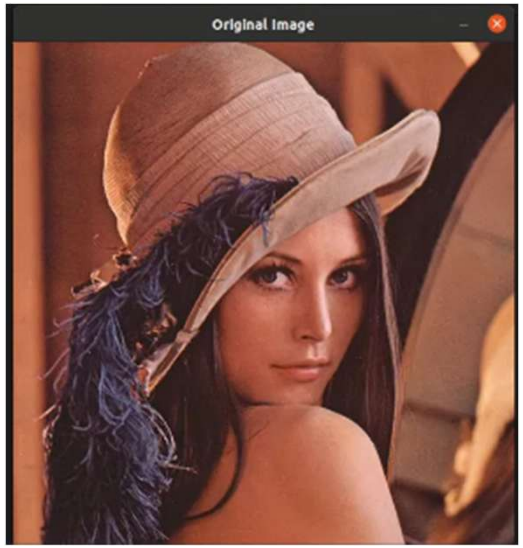
src > opencv_example > src > test.py

```python
#!/usr/bin/env python3
import os
import cv2
import numpy as np

# 이미지 경로 설정
image_path = '/home/ubuntu/catkin_ws/src/opencv_example/src/image/test.jpg'
image = cv2.imread(image_path)

# 원본 이미지를 화면에 출력
cv2.imshow('Original Image', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

UNiCON
Unmanned & Intelligent systems CONtrol LAB

1) 이미지 출력

- **Python 환경에서 Open CV를 통해 다운받은 원본 이미지를 화면에 출력하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.



UNiCON
Unmanned & Intelligent systems CONtrol LAB

2) 이미지 gray scale

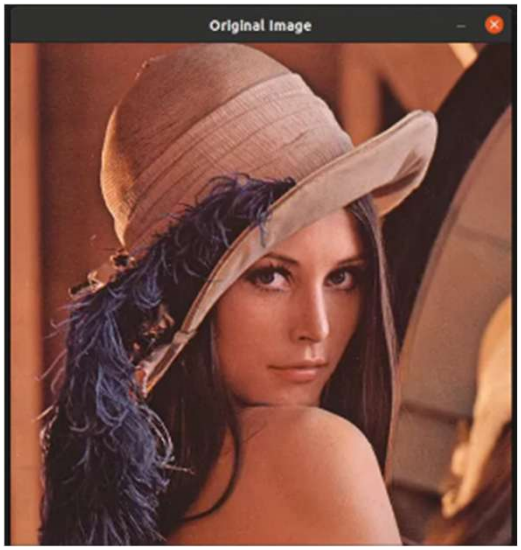
- **Python 환경에서 Open CV를 통해 원본 이미지를 gray scale화하여 화면에 출력하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.

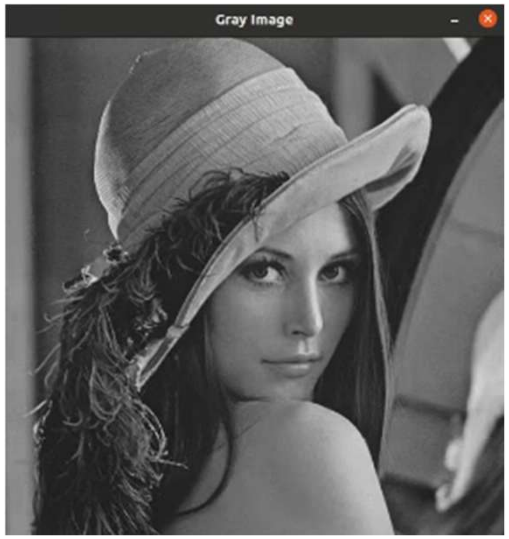
src > opencv_example > src > test.py

```python
#!/usr/bin/env python3
import os
import cv2
import numpy as np

# 이미지 경로 설정
image_path = '/home/ubuntu/catkin_ws/src/opencv_example/src/image/test.jpg'
image = cv2.imread(image_path)

# 이미지 Gray scale화 수행
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Grayscale 화면에 출력
cv2.imshow('Gray Image', gray_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

2) 이미지 gray scale

- **Python 환경에서 Open CV를 통해 원본 이미지를 gray scale화하여 화면에 출력하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.





UNiCON
Unmanned & Intelligent systems CONtrol LAB

3) 이미지 edge 검출

- **Python 환경에서 Open CV를 통해 원본 이미지의 Edge를 검출하여 화면에 출력하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.

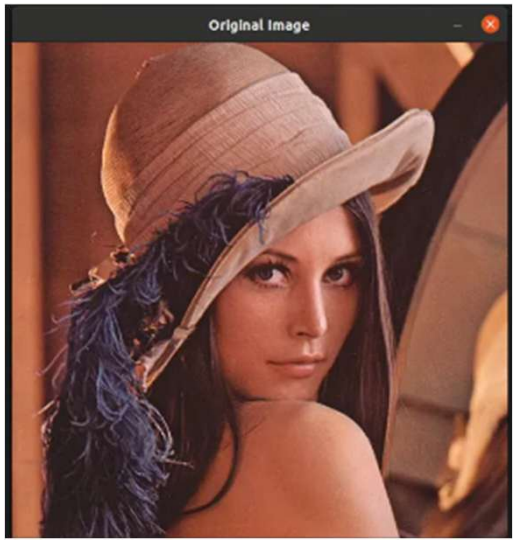
src > opencv_example > src > test.py

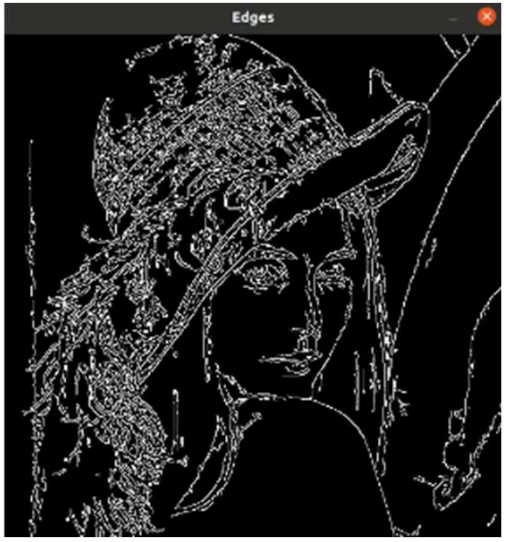
```python
#!/usr/bin/env python3
import os
import cv2
import numpy as np

# 이미지 경로 설정
image_path = '/home/ubuntu/catkin_ws/src/opencv_example/src/image/test.jpg'
image = cv2.imread(image_path)

# 이미지의 Edge detection 수행
edges = cv2.Canny(image, 50, 150)

# 원본 이미지 내 Edge를 화면에 출력
cv2.imshow('Edges', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

UNiCON
Unmanned & Intelligent systems CONtrol LAB

3) 이미지 edge 검출

- **Python 환경에서 Open CV를 통해 원본 이미지의 Edge를 검출하여 화면에 출력하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.





UNiCON
Unmanned & Intelligent systems CONtrol LAB

4) 이미지 윤곽선 검출

- **Python 환경에서 Open CV를 통해 원본 이미지의 윤곽선을 검출하여 화면에 출력하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.

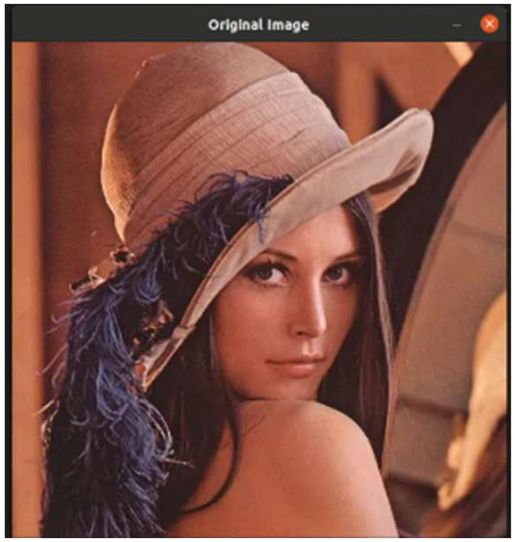
src > opencv_example > src > test.py

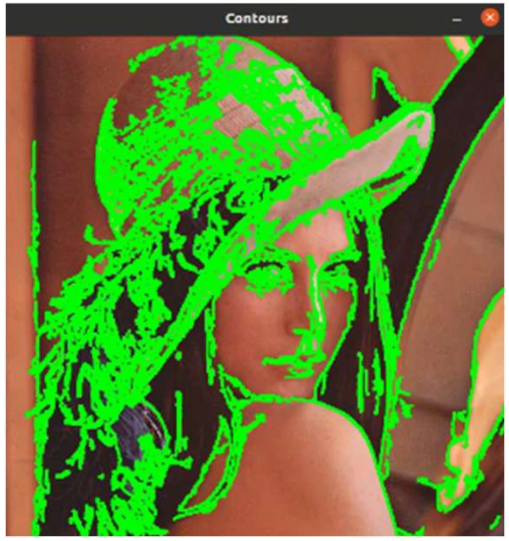
```python
#!/usr/bin/env python3
import os
import cv2
import numpy as np

image_path = '/home/ubuntu/catkin_ws/src/opencv_example/src/image/test.jpg'
image = cv2.imread(image_path)

edges = cv2.Canny(image, 50, 150)

contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
contour_image = image.copy()
cv2.drawContours(contour_image, contours, -1, (0, 255, 0), 2)

cv2.imshow('Contours', contour_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

UNiCON
Unmanned & Intelligent systems CONtrol LAB

4) 이미지 윤곽선 검출

- **Python 환경에서 Open CV를 통해 원본 이미지의 윤곽선을 검출하여 화면에 출력하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.





UNiCON
Unmanned & Intelligent systems CONtrol LAB

5) 이미지 회전 / 크기 조절 / 기울이기

- **Python 환경에서 Open CV를 원본 이미지를 변환(회전, 크기 조절, 기울이기) 후 출력하기 (실습)**

  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.

  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.

src > opencv_example > src > test.py

```python
#!/usr/bin/env python3
import os
import cv2
import numpy as np

image_path = '/home/ubuntu/catkin_ws/src/opencv_example/src/image/test.jpg'
image = cv2.imread(image_path)

# 이미지 회전
(h, w) = image.shape[:2] # (height, width, channel) = image.shape[:]
center = (w / 2, h / 2)
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated_image = cv2.warpAffine(image, M, (w, h))
cv2.imshow('Rotated Image', rotated_image)

# 이미지 크기 조정
resized_image = cv2.resize(image, (300, 300))
cv2.imshow('Resized Image', resized_image)

# 이미지 기울이기 (Affine Transform)
pts1 = np.float32([[50, 50], [200, 50], [50, 200]])
pts2 = np.float32([[10, 100], [200, 50], [100, 250]])
M = cv2.getAffineTransform(pts1, pts2)
affine_image = cv2.warpAffine(image, M, (w, h))
cv2.imshow('Affine Transformed Image', affine_image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

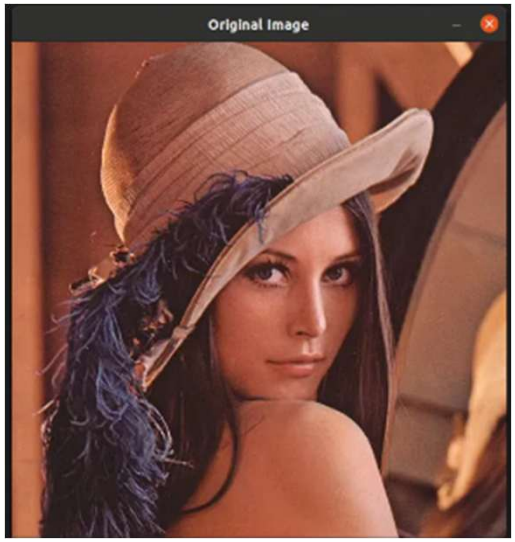
UNiCON
Unmanned & Intelligent systems CONtrol LAB

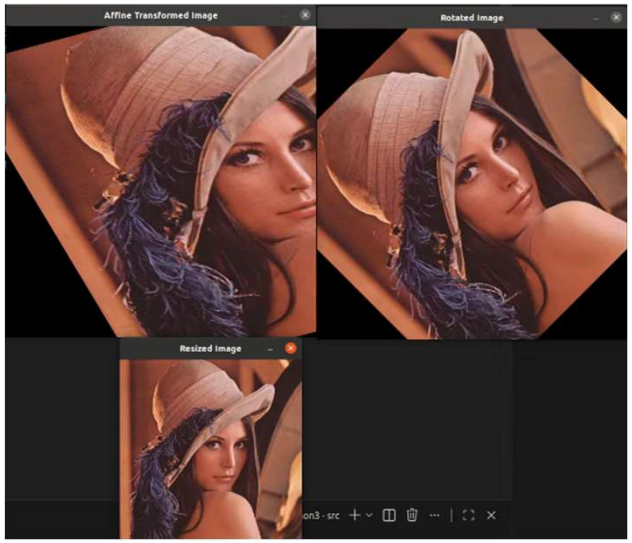
5) 이미지 회전 / 크기 조절 / 기울이기

- **Python 환경에서 Open CV를 원본 이미지를 변환(회전, 크기 조절, 기울이기) 후 출력하기 (실습)**

  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.

  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.





UNiCON
Unmanned & Intelligent systems CONtrol LAB

6) 이미지 filtering

- **Python 환경에서 Open CV를 통해 원본 이미지를 Filtering(Gaussian blur)하여 출력하기 (실습)**

  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.

  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.
  → 주의 사항 3 : kernel 크기는 짝수여야한다.

src > opencv_example > src > test.py

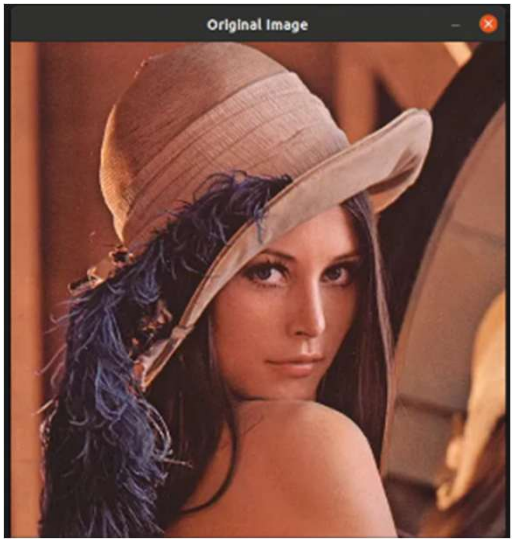
```python
#!/usr/bin/env python3
import os
import cv2
import numpy as np

# 이미지 경로 설정
image_path = '/home/ubuntu/catkin_ws/src/opencv_example/src/image/test.jpg'
image = cv2.imread(image_path)

# Gaussian blur를 통해 이미지 blur 처리 수행 (x-axis kernel, y-axis kernel), sigma
blurred_image = cv2.GaussianBlur(image, (9, 9), 0)


cv2.imshow('Gaussian Blurred Image', blurred_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

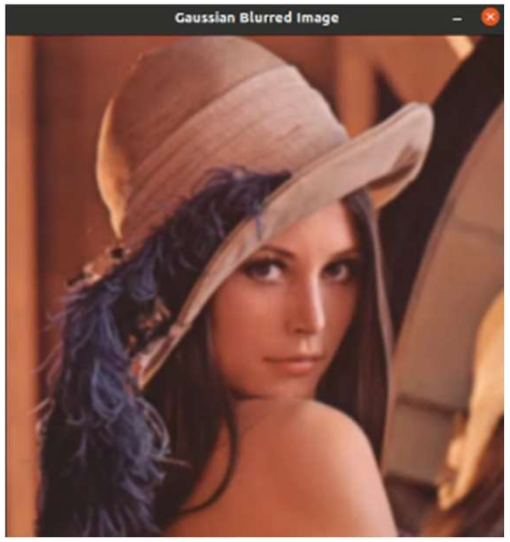
6) 이미지 filtering

- **Python 환경에서 Open CV를 통해 원본 이미지를 Filtering(Gaussian blur)하여 출력하기 (실습)**

  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.

  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.





UNiCON
Unmanned & Intelligent systems CONtrol LAB

7) 이미지 속 객체 검출

- **Python 환경에서 Open CV를 이용하여, 원본 이미지 속 객체를 인식하도록 하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.

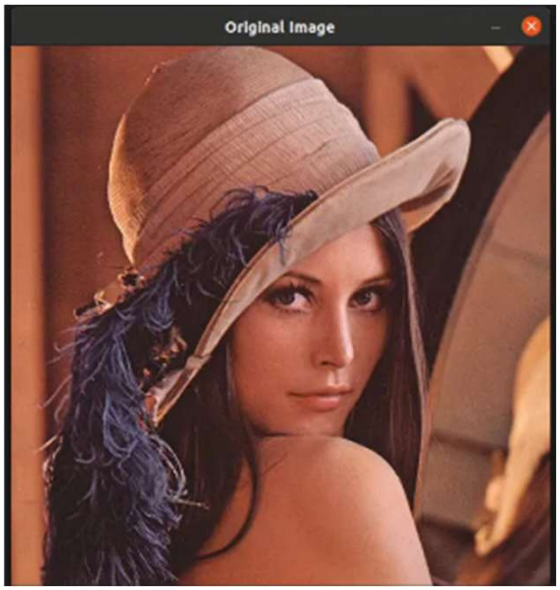
src > opencv_example > src > test.py

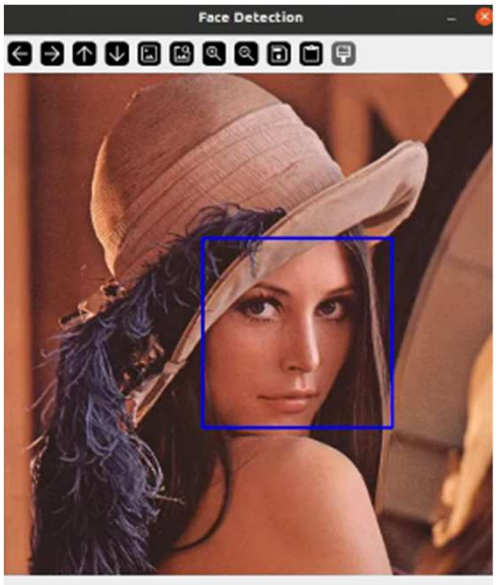
```python
#!/usr/bin/env python3
import os
import cv2
import numpy as np

# 이미지 경로 설정
image_path = '/home/ubuntu/catkin_ws/src/opencv_example/src/image/test.jpg'
image = cv2.imread(image_path)

# Face Cascade(haarcascades)를 통해 이미지 속 사람의 얼굴을 검출
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
faces = face_cascade.detectMultiScale(image, 1.3, 5)
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x+w, y+h), (255, 0, 0), 2)

cv2.imshow('Face Detection', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

UNiCON
Unmanned & Intelligent systems CONtrol LAB

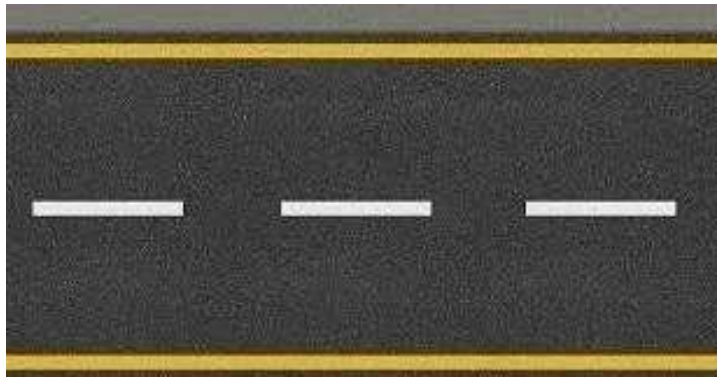
7) 이미지 속 객체 검출

- **Python 환경에서 Open CV를 이용하여, 원본 이미지 속 객체를 인식하도록 하기 (실습)**
  → 주의 사항 1 : 이미지 파일의 확장자는 반드시 “jpg” 또는 “png” 파일을 사용할 것.
  → 주의 사항 2 : Python code 상에 작성하는 이미지의 경로를 정확하게 작성할 것.



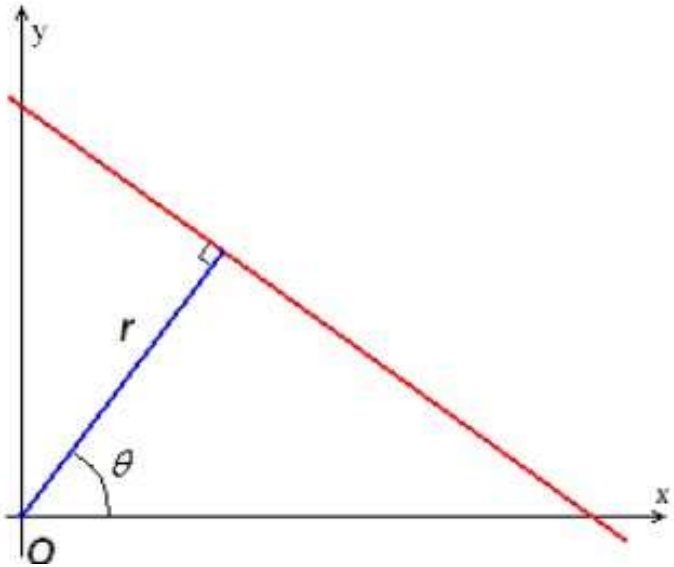


8) 차선 인식 (Hough / HSV / Hough + HSV)



- **위 도로 이미지 속 차선을 Hough transform, HSV(Hue + Saturation + Value) 색 공간 등의 Open CV 기반의 기법을 사용하여 검출**

8) 차선 인식 (Hough / HSV / Hough + HSV)



- **Hough transform 기법은 Computer Vision과 Image Process에서 선, 원, 사각형 등의 도형을 검출하는 데 사용되는 기법**
- **직선 검출 과정**
  : 이미지 gray scale → 이미지 필터링(Gaussian Blur) → 이미지 Edge 검출 → Hough transform

UNiCON
Unmanned & Intelligent systems CONtrol LAB

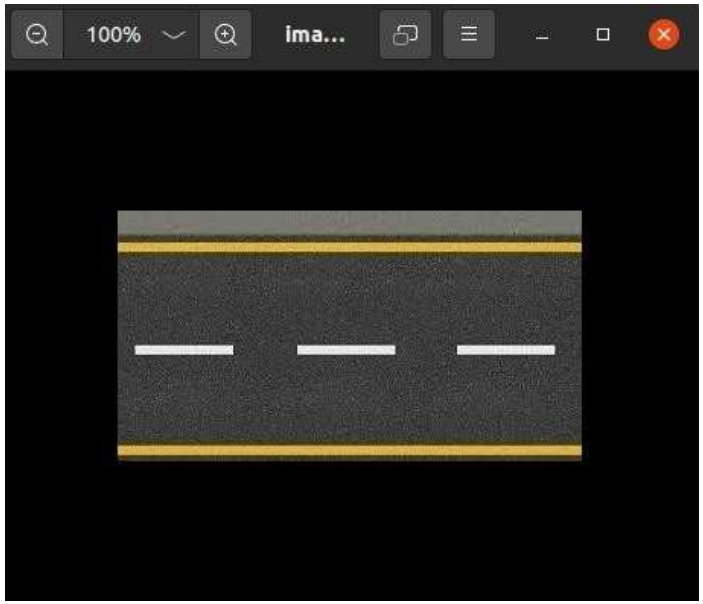
8) 차선 인식 (Hough / HSV / Hough + HSV)

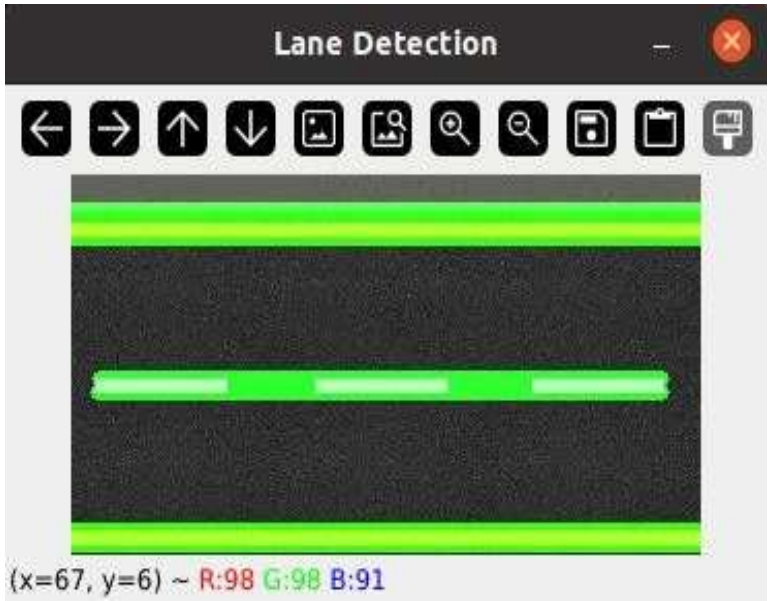
- **Python 환경에서 Open CV의 내장 함수를 이용하여 도로 이미지 속 차선 검출 수행하기 (실습)**
  → 주의 사항 1 : Gray scale, Gaussian blur(filtering), Edge detection, Hough transform 사용
  → 주의 사항 2 : 도로 이미지에 따라 검출 결과의 정확도가 다를 수 있음.

```python
def process_image(image_path):
    # 이미지 읽기
    cv_image = cv2.imread(image_path)

    # 1. 이미지를 회색조로 변환
    gray_image = cv2.cvtColor(cv_image, cv2.COLOR_BGR2GRAY)

    # 2. 가우시안 블러 필터 적용
    blurred_image = cv2.GaussianBlur(gray_image, (5, 5), 0)

    # 3. 에지 검출
    edges = cv2.Canny(blurred_image, 50, 150)

    # 4. Hough 변환을 사용한 선 검출
    lines = cv2.HoughLinesP(edges, 1, np.pi / 180, 20, minLineLength=30, maxLineGap=200)
    line_image = np.zeros_like(cv_image)
    if lines is not None:
        for line in lines:
            x1, y1, x2, y2 = line[0]
            cv2.line(line_image, (x1, y1), (x2, y2), (0, 255, 0), 5)

    # 5. 원본 이미지와 검출된 선을 합성
    combined_image = cv2.addWeighted(cv_image, 0.8, line_image, 1, 1)

    # 결과 이미지 표시
    cv2.imshow('Lane Detection', combined_image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

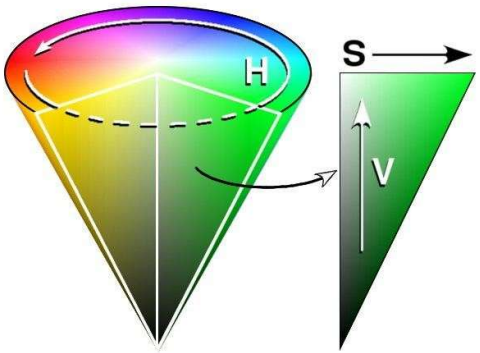
8) 차선 인식 (Hough / HSV / Hough + HSV)

- **Python 환경에서 Open CV의 내장 함수를 이용하여 도로 이미지 속 차선 검출 수행하기 (실습)**
  → 주의 사항 1 : Gray scale, Gaussian blur(filtering), Edge detection, Hough transform 사용
  → 주의 사항 2 : 도로 이미지에 따라 검출 결과의 정확도가 다를 수 있음.
  → **다른 터미널에 ROSCORE 실행시킬 것.**





8) 차선 인식 (Hough / HSV / Hough + HSV)



- **HSV(Hue + Saturation + Value) 모델은 인간의 색 인지에 기반을 둔 색상 모델**
- **HSV는 Hue(색상), Saturation(채도), Value(명도)를 기반으로 하는 색 공간**
- **특정 색상 범위를 쉽게 지정하여 필터링(filtering)할 수 있다는 이점 존재**

UNiCON
Unmanned & Intelligent systems CONtrol LAB

8) 차선 인식 (Hough / HSV / Hough + HSV)

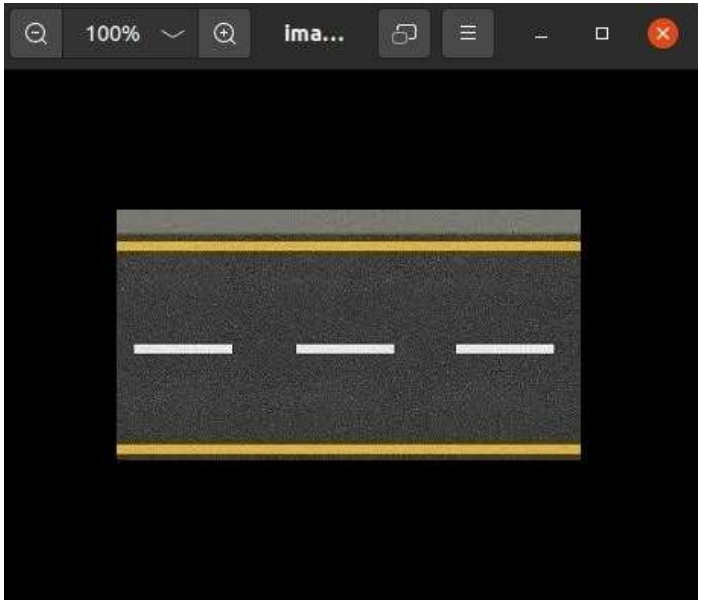
- **Python 환경에서 Open CV의 내장 함수를 이용하여 도로 이미지 속 차선 검출 수행하기 (실습)**
  → 주의 사항 1 : HSV 색상 공간을 사용하여 도로 이미지 속 차선(노란색 차선, 흰색 차선)을 검출할 것.
  → 주의 사항 2 : 도로 이미지에 따라 검출 결과의 정확도가 다를 수 있음.
- **TIP : https://ko.rakko.tools/tools/30/**

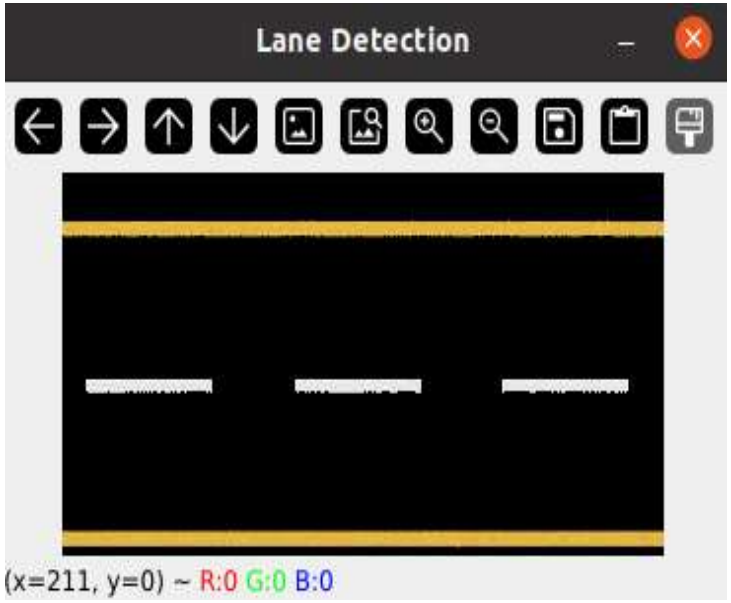
```python
def process_image(image_path):
    # 이미지 읽기
    cv_image = cv2.imread(image_path)

    # 1. 이미지를 HSV 색 공간으로 변환
    hsv_image = cv2.cvtColor(cv_image, cv2.COLOR_BGR2HSV)

    # 2. 노란색 차선 필터링 (HSV 범위 설정)
    yellow_lower = np.array([20, 100, 100])
    yellow_upper = np.array([30, 255, 255])
    yellow_mask = cv2.inRange(hsv_image, yellow_lower, yellow_upper)

    # 3. 흰색 차선 필터링 (HSV 범위 설정)
    white_lower = np.array([0, 0, 200])
    white_upper = np.array([180, 25, 255])
    white_mask = cv2.inRange(hsv_image, white_lower, white_upper)

    # 4. 노란색과 흰색 마스크 결합
    combined_mask = cv2.bitwise_or(yellow_mask, white_mask)

    # 5. 결과 이미지
    combined_image = cv2.bitwise_and(cv_image, cv_image, mask=combined_mask)

    # 결과 이미지 표시
    cv2.imshow('Lane Detection', combined_image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

UNiCON
Unmanned & Intelligent systems CONtrol LAB

8) 차선 인식 (Hough / HSV / Hough + HSV)

- **Python 환경에서 Open CV의 내장 함수를 이용하여 도로 이미지 속 차선 검출 수행하기 (실습)**
  → 주의 사항 1 : HSV 색상 공간을 사용하여 도로 이미지 속 차선(노란색 차선, 흰색 차선)을 검출할 것.
  → 주의 사항 2 : 도로 이미지에 따라 검출 결과의 정확도가 다를 수 있음.





UNiCON
Unmanned & Intelligent systems CONtrol LAB

8) 차선 인식 (Hough / HSV / Hough + HSV)

Method	Characteristic
Hough	▪ Gray scale로 인한 색상 정보 손실 ▪ Edge 검출 수행 시, noise에 민감
HSV	▪ 조명 변화에 민감 ▪ 차선 검출 시, HSV 범위 설정에 민감
Hough + HSV	▪ Noise 및 조명 변화에 대한 강건성 증가 ▪ 다른 객체와의 혼동 감소

UNiCON
Unmanned & Intelligent systems CONtrol LAB

8) 차선 인식 (Hough / HSV / Hough + HSV)

- **Python 환경에서 Open CV의 내장 함수를 이용하여 도로 이미지 속 차선 검출 수행하기 (실습)**
  → 주의 사항 1 : Hough transform과 HSV 색상 공간을 모두 사용하여 이미지 속 차선을 검출할 것.
  → 주의 사항 2 : 도로 이미지에 따라 검출 결과의 정확도가 다를 수 있음.

```python
# 1. 관심 영역(ROI) 설정 (하단 절반)
roi_vertices = [
    (0, height),
    (0, height // 2),
    (width, height // 2),
    (width, height)
]
mask = np.zeros_like(cv_image)
match_mask_color = (255,) * cv_image.shape[2]
cv2.fillPoly(mask, np.array([roi_vertices], np.int32), match_mask_color)
masked_image = cv2.bitwise_and(cv_image, mask)
```

ROI(Region of Interest)를 통해 이미지 속 필요한 영역만을 선택 후 처리 가능

```python
# 2. 이미지를 HSV 색 공간으로 변환
hsv_image = cv2.cvtColor(masked_image, cv2.COLOR_BGR2HSV)

# 3. 노란색 차선 필터링 (HSV 범위 설정)
yellow_lower = np.array([20, 100, 100])
yellow_upper = np.array([30, 255, 255])
yellow_mask = cv2.inRange(hsv_image, yellow_lower, yellow_upper)

# 4. 흰색 차선 필터링 (HSV 범위 설정)
white_lower = np.array([0, 0, 200])
white_upper = np.array([180, 25, 255])
white_mask = cv2.inRange(hsv_image, white_lower, white_upper)

# 5. 노란색과 흰색 마스크 결합
combined_mask = cv2.bitwise_or(yellow_mask, white_mask)

# 6. 에지 검출 (HSV 필터링 후)
edges = cv2.Canny(combined_mask, 50, 150)

# 7. Hough 변환을 사용한 선 검출
lines = cv2.HoughLinesP(edges, 1, np.pi / 180, 20, minLineLength=30, maxLineGap=200)
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(cv_image, (x1, y1), (x2, y2), (0, 255, 0), 5)

# 결과 이미지 표시
cv_image = cv2.resize(cv_image, (960, 540))  # 이미지 크기 조정
cv2.imshow('Lane Detection', cv_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

8) 차선 인식 (Hough / HSV / Hough + HSV)

- **Python 환경에서 Open CV의 내장 함수를 이용하여 도로 이미지 속 차선 검출 수행하기 (실습)**
  → **주의 사항 1 : Hough transform**과 **HSV 색상** 공간을 모두 사용하여 **이미지 속 차선을 검출**할 것.
  → **주의 사항 2 : 도로 이미지에 따라 검출 결과의 정확도가 다를 수 있음.**

9) ROS Gazebo 시뮬레이션 기반 차선 인식

https://www.youtube.com/watch?v=5vF-1hNeew0

9) ROS Gazebo 시뮬레이션 기반 차선 인식

- **ROS Gazebo simulation environment setup**

  → **$ cd ~/catkin_ws/src/**

  → **$ git clone** https://github.com/gihoonbackend/ackermann_vehicle.git

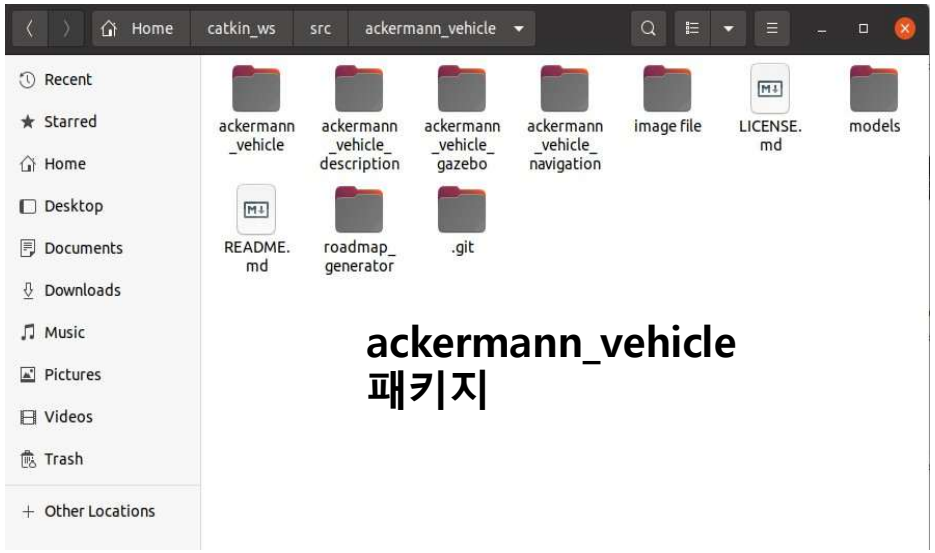
  → **$ sudo apt install ros-noetic-ackermann-msgs**

  → **$ cd ..**

  → **$ rosdep install --from-paths src --ignore-src -r –y**

  → **$ catkin_make**

  → **$ source devel/setup.bash**



UNiCON
Unmanned & Intelligent systems CONtrol LAB

9) ROS Gazebo 시뮬레이션 기반 차선 인식

- **ROS Gazebo simulation environment setup**

→ $ gedit ~/.bashrc

```
alias eb='nano ~/.bashrc'
alias sb='source ~/.bashrc'
alias gs='git status'
alias gp='git pull'
alias cw='cd ~/catkin_ws'
alias cs='cd ~/catkin_ws/src'
alias cm='cd ~/catkin_ws && catkin_make'
source /opt/ros/noetic/setup.bash
source ~/catkin_ws/devel/setup.bash
export ROS_MASTER_URI=http://localhost:11311
export ROS_HOSTNAME=localhost
export TURTLEBOT3_MODEL=burger
export GAZEBO_MODEL_PATH=$GAZEBO_MODEL_PATH:/home/gihoon/catkin_ws/src/ackermann_vehicle/roadmap_generator
```

→ $ export GAZEBO_MODEL_PATH=$GAZEBO_MODEL_PATH

:/home/user/catkin_ws/src/ackermann_vehicle/roadmap_generator

본인의 경로를 정확하게 작성 !

→ $ source ~/.bashrc

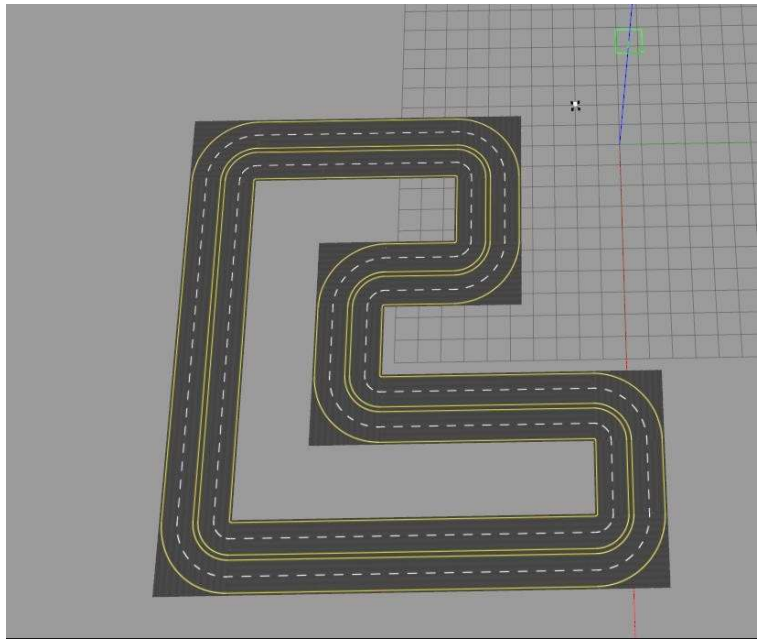
9) ROS Gazebo 시뮬레이션 기반 차선 인식

- **ROS Gazebo simulation environment setup**

  → $ cd catkin_ws/src/ackermann_vehicle/ackermann_vehicle_gazebo/launch

  → $ roslaunch ackermann_vehicle_noetic.launch

  → 위 명령어를 입력하면, Gazebo simulation world와 Robot model이 생성됨



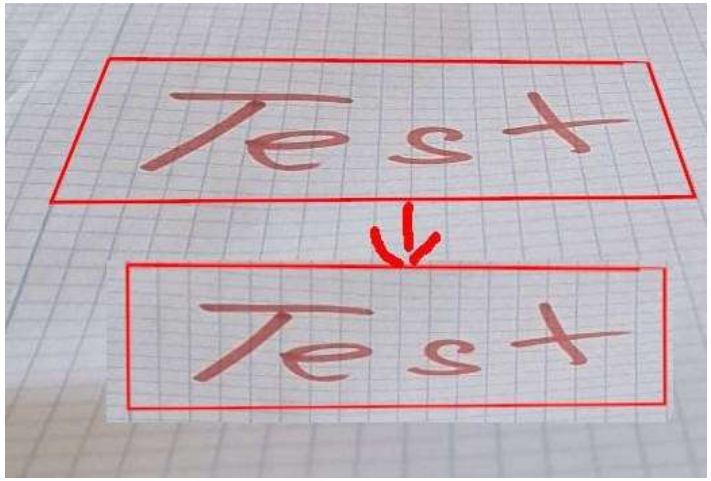
9) ROS Gazebo 시뮬레이션 기반 차선 인식

- **Bird-Eye View**

→ **Bird-Eye View(BEV)**란 로봇이나 차량이 마치 하늘에서 내려다보는 것처럼 도로와 주변 환경을 평평하게 볼 수 있도록 하는 변환 기법

→ **BEV 변환**은 자율주행 및 로봇 공학에서 주로 사용

→ **BEV**는 실 세계의 좌표를 이미지 평면에 정확히 투영하므로, 실제 거리와 각도를 계산할 때 더 정확하게 제공 가능



UNiCON
Unmanned & Intelligent systems CONtrol LAB

9) ROS Gazebo 시뮬레이션 기반 차선 인식

- **Bird-Eye View**

$$w\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

→ **Bird-Eye View(BEV)**는 Open CV의 내장 함수를 통해 구현 가능

→ **Open CV의 getPerspectiveTransform() 함수는 변환 전과 후를 짝짓는 4개의 매핑 좌표를 통해 원근 변환에 필요한 3x3 크기의 원근 변환 행렬을 계산하여 도출**

→ **Open CV의 warpPerspective() 함수는 getPerspectiveTransform() 함수를 통해 계산된 3x3 크기의 원근 변환 행렬을 기반으로 특정 이미지에 대한 원근 변환을 수행**

9) ROS Gazebo 시뮬레이션 기반 차선 인식

- **ROS Gazebo simulation 환경에서 Camera image 데이터를 Subscribe한 뒤, Camera image 속 흰색 차선과 노란색 차선을 인식하기 (실습)**

→ **주의 사항 1 :** 차량(로봇) 모델을 Gazebo simulation 환경 속 도로로 이동 후 명령어를 입력할 것.

→ **주의 사항 2 :** 앞서 설명한 Gazebo simulation 환경을 먼저 실행 후, 새로운 터미널에서 실행할것.

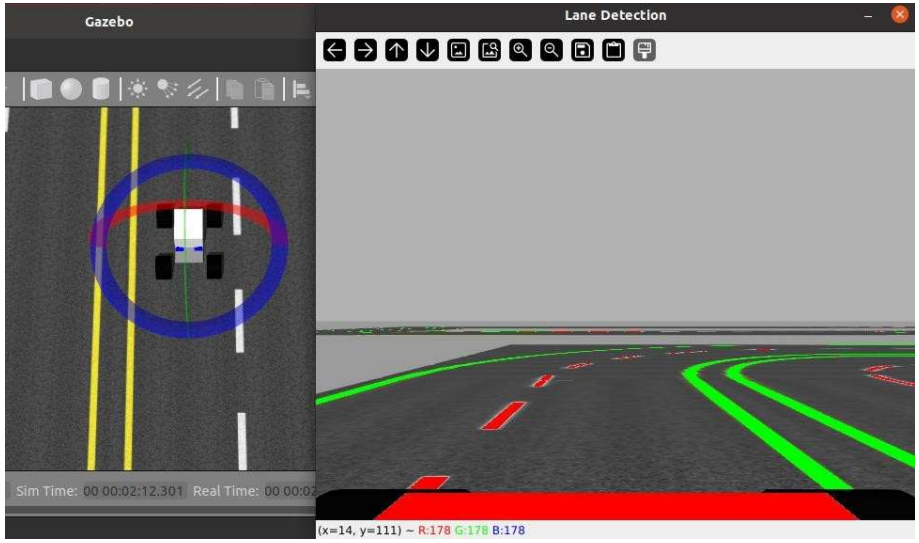
UNiCON
Unmanned & Intelligent systems CONtrol LAB

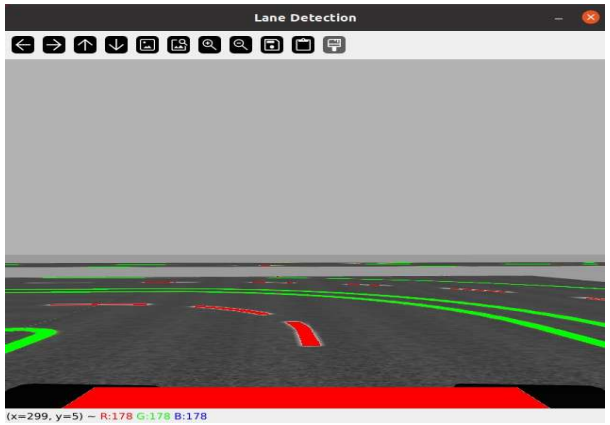
9) ROS Gazebo 시뮬레이션 기반 차선 인식

```python
def camera_callback(self, data):
    try:
        # Read camera data
        cv_image_raw = self.bridge.imgmsg_to_cv2(data, "bgr8")

        hsv = cv2.cvtColor(cv_image_raw, cv2.COLOR_BGR2HSV)

        # White mask HSV range
        lower_white = np.array([0, 0, 180])
        upper_white = np.array([255, 30, 255])

        # Yellow mask HSV range
        lower_yellow = np.array([20, 100, 100])
        upper_yellow = np.array([30, 255, 255])

        mask_white = cv2.inRange(hsv, lower_white, upper_white)
        mask_yellow = cv2.inRange(hsv, lower_yellow, upper_yellow)

        # Find white and yellow lane points
        white_lane_points = cv2.findNonZero(mask_white)
        yellow_lane_points = cv2.findNonZero(mask_yellow)

        # Draw white lane points as red dots
        if white_lane_points is not None:
            for point in white_lane_points:
                x, y = point[0]
                cv_image_raw[y, x] = [0, 0, 255]  # Red color

        # Draw yellow lane points as green dots
        if yellow_lane_points is not None:
            for point in yellow_lane_points:
                x, y = point[0]
                cv_image_raw[y, x] = [0, 255, 0]  # Green color

        cv2.imshow('Lane Detection', cv_image_raw)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            rospy.signal_shutdown("User exit with 'q' key")
            cv2.destroyAllWindows()

    except CvBridgeError as e:
        print(e)
```

- **Camera 센서로부터 이미지 데이터 읽기**
  → ROS 이미지 메시지를 Open CV 이미지로 변환
- **HSV 색상 공간 변환**
  → BGR 이미지를 HSV 색상 공간으로 변환
- **흰색과 노란색 mask 생성**
  → HSV 색상 범위를 사용하여 흰색과 노란색 mask 생성
- **차선 포인트 찾기 및 차선 그리기**
  → cv2.findNonZero() 함수를 통해 포인트를 찾고 표시
- **이미지 화면에 출력**
  → Open CV 윈도우에 변환한 이미지를 출력

9) ROS Gazebo 시뮬레이션 기반 차선 인식





→ 다음의 그림과 같이
도로를 인식하면 성공

9) ROS Gazebo 시뮬레이션 기반 차선 인식

- **ROS Gazebo simulation 환경에서 Camera image 데이터를 Subscribe한 뒤, Camera image를 "Bird-Eye View"로 변환 후 흰색 차선을 인식 후 및 따라가기**

  (실습)

  → **주의 사항 1 :** 차량(로봇) 모델을 Gazebo simulation 환경 속 도로로 이동 후 명령어를 입력할 것.

  → **주의 사항 2 :** 앞서 설명한 Gazebo simulation 환경을 먼저 실행 후, 새로운 터미널에서 실행할 것.

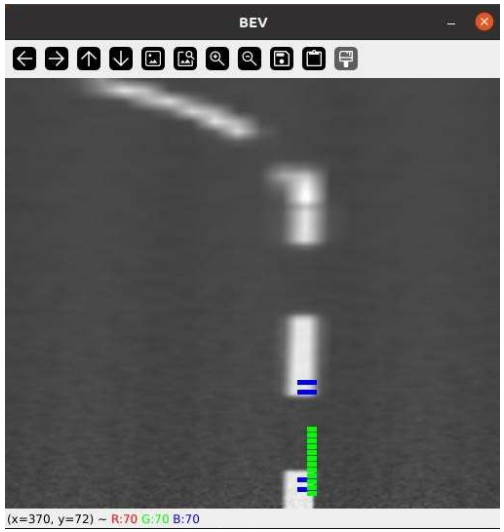
  → **주의 사항 3 :** 인식된 흰색 차선에 로봇이 추종해야 할 way point가 표시될 수 있도록 할 것.

UNiCON
Unmanned & Intelligent systems CONtrol LAB

9) ROS Gazebo 시뮬레이션 기반 차선 인식

```python
def camera_setting(self, data):
    # read camera data, and ROI
    cv_image_raw = self.bridge.imgmsg_to_cv2(data, "bgr8")
    cv_image = cv_image_raw[0:640][300:480]

    # bev_tf
    A = [(251, 13), (400, 13), (580, 140), (70, 140)] # two line
    bev_image = self.bev.birdeyeview(cv_image, A)

    hsv = cv2.cvtColor(bev_image, cv2.COLOR_BGR2HSV)

    # white mask hsv range
    lower_white = np.array([0, 0, 200])
    upper_white = np.array([180, 25, 255])

    mask_white = cv2.inRange(hsv, lower_white, upper_white)

    image = bev_image.copy()

    return mask_white, image
```

- **Camera 센서로부터 이미지 데이터 읽기**
  → ROS 이미지 메시지를 Open CV 이미지로 변환
- **이미지 데이터의 관심 영역 설정**
  → ROI(Region of Interest)를 설정
- **ROI가 설정된 이미지의 시점 변환**
  → 변환에 사용될 4개의 점을 기반으로 Bird-Eye View(BEV) 변환 수행
- **이미지 데이터의 색상 공간 변환**
  → BGR 이미지를 HSV 색상 공간으로 변환



9) ROS Gazebo 시뮬레이션 기반 차선 인식

- **ROS Gazebo simulation 환경에서 Camera image 데이터를 Subscribe한 뒤, image를 "Bird-Eye View"로 변환 후 흰색 차선과 노란색 차선을 인식 후 주행하기**

(실습)

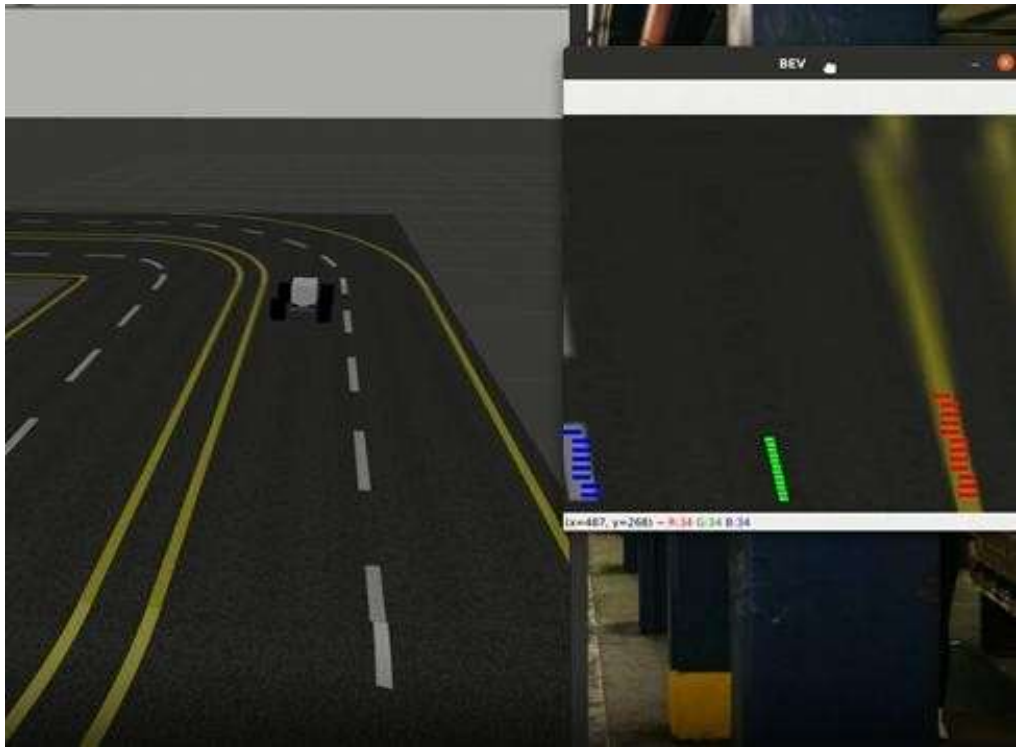
→ **주의 사항 1 :** 차량(로봇) 모델을 Gazebo simulation 환경 속 도로로 이동 후 명령어를 입력할 것.

→ **주의 사항 2 :** 앞서 설명한 Gazebo simulation 환경을 먼저 실행 후, 새로운 터미널에서 실행할 것.

→ **주의 사항 3 :** 인식된 흰색 차선(왼쪽)과 노란색 차선(오른쪽)의 중간점을 추출하여 추종할 수 있도록 알고리즘을 구현할 것.

UNiCON
Unmanned & Intelligent systems CONtrol LAB

9) ROS Gazebo 시뮬레이션 기반 차선 인식



UNiCON
Unmanned & Intelligent systems CONtrol LAB

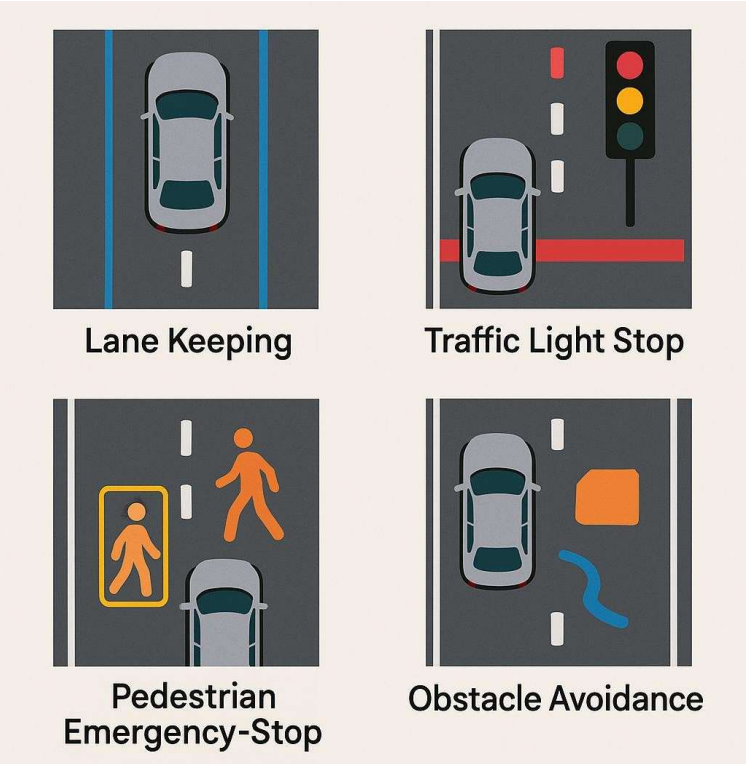
실 습

✓ 현재 시뮬레이션 기반 자율주행 알고리즘 제작

아이디어 기술적 구현이 힘들다면, 강사 및 조교에게 도움 요청

예시

- ➔ **차선 이탈 방지**
- ➔ **커브 구간 자동 감속**
- ➔ **긴급정지**
- ➔ **차선 변경**



QnA

E-mail : phj990608@hanyang.ac.kr

Mobile : 010-7282-0961